

Spec-Driven Development with OpenSpec and GitHub Copilot

A Practical Guide for Java and Angular Developers

Gregg Obst

09/20/2026

Table of contents

Preface	1
I. Part I: Why Durable Intent Matters	3
1. From Chat Sessions to Durable Intent	5
1.1. Learning objective	5
1.2. How SDD and OpenSpec help	5
1.3. The coordination problem	5
1.4. Jira, chat, and repository artifacts	5
1.5. When a specification earns its cost	6
1.6. Running example	6
1.7. Hands-on exercise	6
1.8. Verification checkpoint	6
1.9. What you learned	7
1.10. Online references	7
2. The OpenSpec Operating Model	9
2.1. Learning objective	9
2.2. The operating model	9
2.3. Repository anatomy	9
2.4. Commands and workflows	9
2.5. Hands-on exercise	9
2.6. Verification checkpoint	10
2.7. What you learned	10
2.8. Online references	10
3. Writing Requirements an Agent Can Test	11
3.1. Learning objective	11
3.2. How SDD supports the objective	11
3.3. From acceptance criteria to requirements	11
3.4. Scenario design	12
3.5. Tests can prove the wrong thing	12
3.6. Hands-on exercise	12
3.7. Verification checkpoint	12
3.8. What you learned	13
3.9. Online references	13
4. Engineering Context for GitHub Copilot	15
4.1. Learning objective	15
4.2. How context engineering supports SDD	15
4.3. Context limits and failure modes	15
4.4. Repository instructions	15

4.5. Prompt files and Agent Skills	15
4.6. Hands-on exercise	15
4.7. Verification checkpoint	16
4.8. What you learned	16
4.9. Online references	16
II. Part II: Establishing the Greenfield Baseline	17
5. Preparing the Two-Repository Lab	19
5.1. Learning objective	19
5.2. How the lab supports the objective	19
5.3. Backend baseline	19
5.4. Frontend baseline	19
5.5. VS Code and IntelliJ IDEA setup	20
5.6. Hands-on exercise	20
5.7. Verification checkpoint	20
5.8. What you learned	20
5.9. Online references	20
6. Initialize and Configure OpenSpec	21
6.1. Learning objective	21
6.2. What initialization adds	21
6.3. Install and inspect the CLI	21
6.4. Configure the backend repository	21
6.5. Configure the frontend repository	21
6.6. Hands-on exercise: initialize both repositories	22
6.7. Verification checkpoint: inspect the result	22
6.8. What you learned	22
6.9. Online references	22
7. Exploring the Unit-Number Search	23
7.1. Learning objective	23
7.2. How exploration supports the objective	23
7.3. Start from the Jira story	23
7.4. Questions that shape the behavior	23
7.5. Compare exploration in both IDEs	24
7.6. Hands-on exercise	24
7.7. Verification checkpoint	25
7.8. What you learned	25
7.9. Online references	25
8. Proposing the First Full-Stack Change	27
8.1. Learning objective	27
8.2. How proposal supports the objective	27
8.3. Start with two linked changes	27
8.4. Backend proposal	28
8.5. Frontend proposal	28
8.6. Order the tasks	29
8.7. Review the artifacts	29
8.8. Hands-on exercise	30

8.9. Verification checkpoint	30
8.10. What you learned	30
8.11. Online references	30
III. Part III: Applying and Evolving the Specification	31
9. Applying the Spring Boot Change	33
9.1. Learning objective	33
9.2. How apply supports the objective	33
9.3. Backend implementation	33
9.4. Contract ownership	34
9.5. Backend tests	34
9.6. Hands-on exercise	34
9.7. Verification checkpoint	34
9.8. What you learned	34
9.9. Online references	35
10. Applying the Angular Change	37
10.1. Learning objective	37
10.2. How the contract supports the objective	37
10.3. Generate the client	37
10.4. Implement the search experience	38
10.4.1. Input and validation	38
10.4.2. Request and display states	38
10.5. Trace scenarios to implementation	39
10.6. Frontend tests	39
10.7. Hands-on exercise	39
10.8. Verification checkpoint	40
10.9. What you learned	40
10.10. Online references	40
11. Verifying, Synchronizing, and Archiving	41
11.1. Learning objective	41
11.2. Four different checks	41
11.3. Validate the artifacts	42
11.4. Verify the implementation	42
11.5. Synchronize accepted changes	42
11.6. Archive only completed work	43
11.7. The fresh-session test	43
11.8. Hands-on exercise	43
11.9. Verification checkpoint	44
11.10. What you learned	44
11.11. Online references	44
12. Adding VIN Search to a Brownfield System	45
12.1. Learning objective	45
12.2. Start with the accepted behavior	45
12.3. Evolve the Angular client	46
12.4. What you learned	47
12.5. Online references	47

IV. Part IV: Change Under Pressure	49
13. Modifying Accepted Behavior	51
13.1. Learning objective	51
13.2. Start with the accepted requirement	51
13.3. Write a complete modification	51
13.4. Full-stack impact	52
13.5. Hands-on exercise	52
13.6. Verification checkpoint	53
13.7. What you learned	53
13.8. Online references	53
14. Revising an In-Flight Change	55
14.1. Learning objective	55
14.2. Start with the decision that changed	55
14.3. Rebuild the active artifact	55
14.4. Decide whether to amend or split the change	56
14.5. Keep implementation and review synchronized	56
14.6. Hands-on exercise	57
14.7. Verification checkpoint	57
14.8. What you learned	57
14.9. Online references	58
15. Coordinating Parallel Changes	59
15.1. Learning objective	59
15.2. Isolation is not compatibility	59
15.3. A pair of truck-search changes	59
15.4. Compare deltas before archive	60
15.5. Coordinate the contract boundary	60
15.6. Reconcile and archive	61
15.7. Hands-on exercise	61
15.8. Verification checkpoint	62
15.9. What you learned	62
15.10. Online references	62
16. Making SDD a Team Practice	63
16.1. Learning objective	63
16.2. Put decisions in the right place	63
16.3. Make traceability recoverable	63
16.4. Assign review ownership	64
16.5. Scale the artifacts to the risk	64
16.6. Maintain the context that agents use	64
16.7. Capstone: defend the amount of specification	65
16.8. Verification checkpoint	65
16.9. What you learned	65
16.10. Online references	65
17. Summary	67
References	69
OpenSpec and workflow	69

Petclinic and technology	69
AI-assisted development	69
Appendices	71
A. Installing and Updating OpenSpec	71
A.1. Scope	71
A.2. Before installation	71
A.3. Install OpenSpec 1.13.1	71
A.4. Inspect the installed version	71
A.5. Upgrade OpenSpec	71
A.6. Update an existing repository	72
A.7. Verification checklist	72
A.8. Online references	72
B. OpenSpec CLI Command Reference	73
B.1. Typical workflow	73
B.2. Core commands	73
B.3. Store and workset commands	74
C. GitHub Copilot Commands and OpenSpec Skills	77
C.1. Using the matrix	78
D. OpenSpec schema.yaml and Template Reference	81
D.1. Schema fields	81
D.2. Template structure	83
E. Troubleshooting OpenSpec	85
E.1. A diagnostic method	85
E.2. Installation failures	85
E.3. Initialization and update failures	85
E.4. Command discovery failures	85
E.5. Artifact and validation failures	86
E.6. Two-repository failures	86
E.7. Recovery boundaries	86
E.8. Online references	86
F. Further Reading and Community Resources	87
F.1. Official documentation	87
F.2. Community schemas and extensions	87
G. Lab Setup	89
G.1. Required tools	89
G.2. Prepare the exercise repositories	89
G.3. Preflight check	90
H. Prompt Workbook	91
H.1. Explore unit-number search	91
H.2. Propose unit-number search	91
H.3. Add VIN search	91

Table of contents

H.4. Modify unit-number validation	91
H.5. Update an active filter proposal	91
H.6. Review generated artifacts	92
H.7. Verify before archive	92
I. Artifact Review Checklists	93
I.1. Proposal	93
I.2. Requirements and scenarios	93
I.3. Design	93
I.4. Tasks	93
I.5. Apply review	94
I.6. Archive readiness	94
J. Instructor Guide	95
J.1. Course shape	95
J.2. Teaching checkpoints	95
J.3. Expected friction	95
J.4. Recovery points	96
J.5. Closing discussion	96

Preface

AI-assisted development with tools like GitHub Copilot can be easy to start and difficult to control. A prompt can produce a plausible implementation in minutes, but plausibility is not the same as correctness. The hard questions remain: What are we building? Which parts of the existing system matter? How will we know that the result matches the intent?

This book is for software engineers, technical leads, and architects who want better answers to those questions without turning every change into a heavyweight ceremony. It introduces Spec-Driven Development (SDD) with OpenSpec as a practical way to make intent explicit, keep requirements close to implementation, and give GitHub Copilot the project context it needs. The examples move from greenfield work to brownfield systems, then through the less glamorous parts of real delivery: drift, verification, archiving, governance, and troubleshooting.

The approach has costs. Writing a useful spec takes time, and a vague or oversized spec can slow a team down instead of helping it. OpenSpec does not remove judgment from engineering, and Copilot can still misunderstand both the code and the specification. The goal is not to add paperwork for its own sake. It is to create a small, inspectable record of intent that helps people and tools make fewer expensive assumptions.

This book presents Spec-Driven Development (SDD) as a practical response to that continuity problem. It uses OpenSpec to keep requirements, proposed changes, design decisions, and implementation tasks in the repository where developers and agents can review them together.

The running example is a commercial truck search application. Its requirements change over three 90-minute mornings: the team first searches by a 15-character unit number, then adds VIN search, then shortens the unit-number limit and revises an active filter proposal. The changes are small enough to follow, but they expose the coordination problems that appear in larger systems.

This is not an argument for writing a specification before every code edit. Small, local, reversible changes may not justify the overhead. The workflow earns its cost when behavior crosses layers, requirements are likely to change, or several people and agent sessions need the same account of what the system should do.

Each part mirrors one morning of the training class. You can read the book independently or use its exercises as the class lab guide.

Part I.

Part I: Why Durable Intent Matters

1. From Chat Sessions to Durable Intent

1.1. Learning objective

By the end of this chapter, you should be able to explain the coordination problem that spec-driven development addresses and decide when a change might warrant a specification.

1.2. How SDD and OpenSpec help

- What would change if agreed behavior moved out of a single Copilot session and into reviewable repository artifacts?
- OpenSpec offers a place for that behavior beside the code, along with a workflow for proposing, applying, and archiving changes. What does that make easier to review later?
- The process does not replace engineering judgment. It records the decisions that judgment produced so the team can revisit them.

1.3. The coordination problem

- Start with a developer who asks Copilot to implement a change and ends the session with plausible code. What does the next person inherit?
- Follow the next developer or agent session, which can see the code but not the discarded alternatives and final decisions from the original conversation.
- The implementation may satisfy a reasonable interpretation while still violating the business intent. How would the team recognize that gap?
- Three things are easy to confuse: what the agent can generate, what the team has agreed to build, and what the tests prove. How might separating them change the conversation?

1.4. Jira, chat, and repository artifacts

- What jobs belong to a Jira story, a chat transcript, code, tests, and a specification?
- Jira can track the request, ownership, priority, and delivery status. OpenSpec can record behavior, proposed change, design decisions, and implementation tasks. Where should each piece of information live?
- Why would neither a Jira story nor a chat transcript replace the other artifacts?
- Issue identifiers and links can add traceability without requiring Jira API automation in this book.

1.5. When a specification earns its cost

- Before choosing the workflow, consider four conditions: change risk, ambiguity, coordination cost, and reversibility.
- A full specification may make sense when behavior crosses the backend and frontend, requirements may change, or multiple developers and agent sessions need the same intent.
- A smaller record or direct edit may be enough for a change that is local, obvious, and easy to reverse.
- There is a real cost: a specification can become ceremony or stale documentation when the team does not review and update it. What review habit would keep that from happening?

1.6. Running example

The book uses a commercial truck search system as its running example:

- The backend uses Java 21 and Spring Boot 4.1.1.
- The frontend uses Angular 21 and TypeScript.
- The backend and frontend live in separate repositories, each with its own OpenSpec history.
- The first request asks for search by a 15-character truck unit number but leaves several behaviors unsettled.

Those missing decisions make the problem concrete: matching rules, invalid input, no-match behavior, response shape, and the boundary between the two repositories.

1.7. Hands-on exercise

Review the unit-number Jira story and list every behavior GitHub Copilot would otherwise have to guess. Could you classify each item as one of the following?

- a business requirement;
- an implementation decision;
- a test or verification question; or
- an explicit non-goal.

1.8. Verification checkpoint

Produce a short decision record containing:

- unresolved questions that block implementation;
- assumptions the team has accepted;
- behaviors outside the first change; and
- an SDD or no-SDD decision with its reasons.

Another developer should be able to understand what remains undecided without reading the original Copilot conversation. Does the record meet that test?

1.9. What you learned

You should now be able to distinguish a work-management request from an engineering specification, identify when Copilot lacks durable context, and judge whether the cost of recording intent is justified. The next chapter turns that intent into OpenSpec artifacts.

1.10. Online references

- [OpenSpec documentation](#)
- [GitHub Copilot documentation](#)

2. The OpenSpec Operating Model

2.1. Learning objective

By the end of this chapter, you should be able to read an OpenSpec repository, decide which artifact fits a piece of information, and explain how accepted behavior can become an implemented and archived change.

2.2. The operating model

- Markdown artifacts can serve as reviewable inputs to agent work.
- Accepted behavior is easier to follow when it stays separate from active and archived changes.
- Dependencies between artifacts can expose missing decisions before implementation begins.
- The repository tree can act as a record of what the team accepts, what it is changing, and what it has completed.

2.3. Repository anatomy

- A useful tour starts with `openspec/specs/`, `openspec/changes/`, `schemas`, `prompts`, and `skills`.
- The rationale and scope generally belong in `proposal.md`, behavioral changes in `delta specs`, implementation decisions in `design.md`, and work items in `tasks.md`.
- Comparing the current source of truth with a proposed delta helps show which behavior stays, changes, or is added.

2.4. Commands and workflows

- It helps to distinguish terminal CLI commands from Copilot slash commands and Agent Skills.
- The workflow moves from `explore` and `propose` through `apply`, `verify`, `sync`, and `archive`.
- Operations may depend on an optional profile, a particular host, or a local environment, so those conditions belong near the operation they affect.

2.5. Hands-on exercise

Try classifying sample statements as rationale, requirement, design decision, implementation task, or verification evidence. For each statement, identify the artifact that would contain it and the consequence of placing it elsewhere.

2.6. Verification checkpoint

The checkpoint is to place each statement in the artifact that fits it and explain why moving it elsewhere could weaken the workflow. A satisfactory classification distinguishes behavior from implementation and evidence from intent.

2.7. What you learned

At this point, you should be able to read an OpenSpec tree as a record of current behavior, intended change, implementation work, and completed history. The next chapter uses those distinctions to introduce requirements and testable scenarios.

2.8. Online references

- [OpenSpec concepts](#)
- [OpenSpec CLI reference](#)
- [OpenSpec skills reference](#)

3. Writing Requirements an Agent Can Test

3.1. Learning objective

Write observable requirements and scenarios that constrain implementation without prescribing its REST endpoints, database queries, or Angular components.

3.2. How SDD supports the objective

An agent can turn an underspecified Jira story into a plausible implementation quickly. That speed can make an unanswered question expensive: the agent might choose a validation rule, a matching strategy, or an error response before anyone notices the decision.

One way to handle that risk is to move those decisions into a reviewable specification before implementation begins. The requirements can describe behavior, the scenarios can make that behavior observable, and the design can record the technical choices needed to implement it. OpenSpec keeps those concerns in separate artifacts, which gives the agent something more stable than a chat transcript to work from.

The specification does not replace judgment. When the team has not decided what should happen, it can be better to preserve that question for review than to hide an agent's guess in code.

3.3. From acceptance criteria to requirements

The running example starts with a Jira request for exact search by a 15-character truck unit number. That sentence identifies the capability, but it does not describe every behavior that implementation and verification need. Before creating the change, ask which inputs are valid, what happens when the input is missing or invalid, what a match returns, and what the user sees when no record matches.

Once a decision is settled, turn it into a normative requirement. SHALL or MUST can mark behavior that is required, while definitions can narrow terms that might otherwise invite different interpretations. For example, the specification needs a shared meaning for 15\characters, exact\match, missing, and invalid.

Keep the requirement at the business-behavior level. It might state that a valid unit number returns matching vehicles without requiring a particular controller method, SQL predicate, repository class, or Angular component. Those choices can live in design.md and in the implementation tasks.

3.4. Scenario design

Given/When/Then scenarios offer one way to turn a requirement into an observable check:

- Given establishes the relevant state and input.
- When names the user or system action.
- Then states the result that a reviewer, test, or running system can observe.

For the unit-number search, scenarios might cover valid input, invalid input, a matching record, and no matching record. A boundary scenario is most useful when the boundary represents required behavior, such as the difference between a 15-character value and a value that exceeds the allowed length.

It is useful to keep the scenarios independent of implementation. A scenario can require that an invalid value is rejected without prescribing whether validation occurs in a controller, service, or Angular form. That separation gives the team room to change the technical design without silently changing the agreed behavior.

3.5. Tests can prove the wrong thing

Green tests provide evidence that an assertion passed. They do not, by themselves, show that the assertion came from an accepted requirement. An agent might write a test around an unsupported assumption, make the implementation satisfy that test, and report success while the product behavior remains wrong.

Before applying the change, trace each scenario to an observable result. During implementation, record which code and test provide that evidence. If a test has no corresponding scenario, ask whether it protects a real requirement or merely reflects an implementation detail. If a scenario has no evidence, the change may not be ready to verify.

3.6. Hands-on exercise

Rewrite the 15-character unit-number story as OpenSpec requirements and Given/When/Then scenarios. Separate the behavior from the technical design. Record unresolved questions rather than choosing answers silently, then review the result with a partner before implementation.

3.7. Verification checkpoint

For each requirement, it helps to check four things:

- The language has one interpretation.
- The behavior can be observed without reading the implementation.
- The requirement stays within the change's scope.
- At least one meaningful scenario provides evidence for it.

Then inspect the scenarios as a set. Together, they can cover the settled valid, invalid, match, and no-match behavior without adding technical choices that the requirements do not need.

3.8. What you learned

A requirement describes behavior the system must provide. A scenario makes one behavior concrete and testable. A design choice explains how the team plans to implement it. A test provides evidence that an implemented behavior produced an expected result. Keeping those roles distinct can give the agent a bounded target and give the team a way to review whether the target was met.

3.9. Online references

- [OpenSpec documentation](#)
- [Spring Boot testing reference](#)
- [Angular testing guide](#)

4. Engineering Context for GitHub Copilot

4.1. Learning objective

Explore how a bounded context package might help GitHub Copilot recover the right constraints across sessions in VS Code and IntelliJ IDEA.

4.2. How context engineering supports SDD

- Store stable repository facts in repository instructions.
- Store repeatable workflows in prompt files and Agent Skills.
- Store change-specific intent in OpenSpec artifacts.
- Consider keeping transient exploration in chat until the team accepts it as part of the project context.

4.3. Context limits and failure modes

- Examine how context-window competition, stale context, accidental omission, and conflicting instructions can affect Copilot's output.
- Consider the difference between adding more context and providing better-scoped context.
- Trace how duplicated rules can drift when they live in multiple files.

4.4. Repository instructions

- Describe the stack, architecture, testing requirements, and generated-code constraints.
- Consider how backend rules might be separated from frontend rules.
- Compare how VS Code and IntelliJ IDEA discover and use repository instructions.

4.5. Prompt files and Agent Skills

- Consider which reusable commands belong in prompt files.
- Consider which multi-step domain workflows belong in Agent Skills.
- Examine how generated OpenSpec integrations work and when `openspec\update` might be appropriate.

4.6. Hands-on exercise

Try breaking one oversized Copilot prompt into stable instructions, reusable workflow guidance, and change-specific artifacts.

4.7. Verification checkpoint

Open a fresh session in each IDE. Check whether the repository can reproduce the required constraints without the earlier transcript.

4.8. What you learned

Decide which durable context source is smallest and most suitable for each kind of information.

4.9. Online references

- [GitHub Copilot customization](#)
- [OpenSpec supported tools](#)
- [OpenSpec skills reference](#)

Part II.

**Part II: Establishing the Greenfield
Baseline**

5. Preparing the Two-Repository Lab

The exercises in Part II use two prepared repositories: `commercial-truck-search-api` and `commercial-truck-search-ui`. The backend owns the OpenAPI contract, and OpenAPI Generator produces the TypeScript Angular client consumed by the frontend. Before asking an agent to change behavior, you may want to establish a baseline for both repositories. That gives you a way to distinguish an environment problem from a specification or implementation defect.

5.1. Learning objective

By the end of this chapter, you should be able to clone, inspect, build, test, and run both repositories. You will also have a record of the tool versions, application ports, and clean commits that can help you repeat the later exercises.

5.2. How the lab supports the objective

- You can use the prepared repositories to keep the exercises focused on SDD rather than project scaffolding.
- You can keep an independent OpenSpec history beside each codebase.
- The backend can remain the API owner while the frontend client is generated from that contract.

5.3. Backend baseline

It may help to inspect the backend before changing it. Check the versions and tools in use: Java 21, Spring Boot 4.1.1, Maven, Spring Data JPA, and H2. Locate the existing test conventions and application boundaries. Also check whether truck search has been implemented. Later chapters use its absence as the starting condition for the first feature.

5.4. Frontend baseline

Inspect the frontend separately. Check for Angular 21 and Node.js 24.19.0, then locate the Vitest, component-test, Playwright, and generated-client boundaries. Record the local backend and frontend ports from the prepared repositories. The two applications may not use the same defaults in every environment.

5.5. VS Code and IntelliJ IDEA setup

Open both repositories in VS Code or IntelliJ IDEA. Check that GitHub Copilot Chat is available in the IDE you plan to use, then record which terminal, task runner, and project view you used for each command. The commands and files remain the same, although each IDE exposes them differently. Keeping that difference visible can help separate an IDE problem from a repository problem.

5.6. Hands-on exercise

Clone `commercial-truck-search-api` and `commercial-truck-search-ui`. Build, test, and run both starter repositories without making a feature change. You can leave OpenSpec uninitialized in this chapter; the next chapter uses the clean baseline to show what initialization adds.

5.7. Verification checkpoint

Record the baseline commands that succeed, along with the tool versions, known application ports, and commit IDs. Before moving on, check that both repositories have passing existing tests and a clean working tree. Those commits can serve as reset points for the later exercises.

5.8. What you learned

A known baseline gives later failures a reference point. If a starter repository does not build or its existing tests do not pass, fix or record that environment problem before an agent implements a feature. That leaves the next chapter free to focus on configuring OpenSpec rather than debugging an unexamined starting state.

5.9. Online references

- Spring Boot system requirements
- Angular local environment setup
- OpenAPI Generator installation

6. Initialize and Configure OpenSpec

6.1. Learning objective

This chapter walks through installing OpenSpec 1.13.1 and configuring both repositories with project context that the team can review and maintain in version control.

6.2. What initialization adds

When you initialize a repository, OpenSpec generates GitHub Copilot integration files from a selected profile. These files can keep repository-specific constraints close to the code they govern and make the configuration visible in version control. That makes them a reasonable place to start, although the generated profile may still need repository-specific review.

6.3. Install and inspect the CLI

You can install the pinned OpenSpec package with npm. It is worth checking that the CLI reports version 1.13.1 and inspecting the available commands before initializing either repository.

It helps to keep two maintenance actions separate:

- Upgrade the package when the team changes the OpenSpec CLI version.
- Run `openspec\update` when the team needs to refresh repository integrations from the installed package.

6.4. Configure the backend repository

In the backend repository, you can initialize GitHub Copilot support with the profile that covers Java, Spring Boot, Maven, JPA, H2, testing, and OpenAPI ownership. Before committing the generated files, review them to see whether the repository-specific rules describe the backend rather than the frontend.

6.5. Configure the frontend repository

In the frontend repository, you can initialize GitHub Copilot support with the profile that covers Angular, the generated client, Vitest, component tests, and Playwright. Backend implementation rules generally belong in the backend repository. The frontend consumes the backend contract, but it does not own the backend implementation.

6.6. Hands-on exercise: initialize both repositories

You can use the parallel VS Code and IntelliJ IDEA instructions to initialize and configure both repositories. Performing the same repository operations in each IDE may help distinguish an IDE integration problem from an OpenSpec configuration problem.

6.7. Verification checkpoint: inspect the result

For a newly initialized repository, consider running the version, status, validation, and health checks that apply. These checks should let you confirm that:

- the CLI reports OpenSpec 1.13.1;
- each repository has the expected generated integration files;
- validation accepts the generated configuration; and
- the health check reports a usable initialization.

Reviewing the generated diff before committing can show what initialization added. The repository instructions can then remain the team-owned description of local constraints.

6.8. What you learned

OpenSpec generates the integration files, while the team can retain ownership of the repository-specific constraints and their review. When the CLI version changes, the team can upgrade the package. When it needs to refresh integrations from the installed version, it can run `openspec\update`.

6.9. Online references

- OpenSpec installation
- OpenSpec CLI reference
- OpenSpec supported tools

7. Exploring the Unit-Number Search

7.1. Learning objective

Exploration can help resolve business uncertainty before implementation artifacts are created. By the end of this chapter, you should have a decision record that describes what the unit-number search may need to do, which questions remain open, and who owns the next answer.

7.2. How exploration supports the objective

Exploration works best as a thinking step rather than an implementation request. The agent can inspect the repositories, compare existing conventions, and expose questions that a short Jira story leaves unstated. A plausible guess should not become a requirement just because it makes the next prompt easier to write.

This distinction can matter for a full-stack change. The backend and frontend will usually need to agree on the meaning of a unit number, the shape of a successful result, and the way failures appear to the user. An unresolved decision in one repository may become a contract mismatch in the other.

7.3. Start from the Jira story

A useful starting point is the request to search by a 15-character unit number. Read the story as a source of stated intent rather than as a complete specification. Extract the acceptance criteria exactly as they are written, then mark any behavior that the story implies but does not define.

It can help to keep the Jira issue connected to the future OpenSpec work. The issue provides the business context; the proposal and specifications can describe the behavior that the team agrees to build. Preserving that link gives reviewers a way to move from the requested change to the artifacts that explain its scope.

During exploration, ask the agent to inspect the relevant backend and frontend conventions. The goal is not to produce `proposal.md`, `design.md`, or `tasks.md` yet. The goal is to discover which decisions those artifacts will have to record.

7.4. Questions that shape the behavior

The phrase “search by a 15-character unit number” may leave several implementation-blocking questions open:

- Does the search require an exact match, or may it return partial matches?
- Does the system normalize input before searching? If so, which characters or formats does it accept?
- What should happen when the input is empty or malformed?
- What should happen when the input is valid but no unit matches it?

7. Exploring the Unit-Number Search

- What fields belong in the result, and how should the frontend display them?
- Which errors belong in the API response, and which should become user-visible messages?
- Which related capabilities are explicitly out of scope for this change?

These questions are better answered with the repository's existing behavior, the business owner's decision, or an explicit deferral than with a guess based on the ticket's wording. A decision record can keep the uncertainty visible:

Question	Decision or status	Owner or next action
Exact or partial matching?	Open	Confirm with the business owner
Empty and malformed input?	Open	Compare existing validation conventions
Unknown unit number?	Open	Define the API and UI behavior together
Result shape?	Open	Agree on the backend-owned contract
Non-goals?	Open	Record deferred search capabilities

The table can be useful even before it contains many answers. It keeps an unanswered question visible instead of allowing it to disappear into generated code, and it gives the eventual proposal a clearer boundary.

7.5. Compare exploration in both IDEs

Running the exploration workflow in VS Code and IntelliJ IDEA can provide a useful comparison. The model may use different wording, organize its findings differently, or notice different repository details in each environment. That variation is not necessarily a problem. The decisions that affect the change still need one agreed record.

Compare the results for coverage rather than prose. Did both explorations identify the same implementation-blocking questions? Did either one infer behavior that the Jira story does not state? Did either one miss a repository convention that the other found? Resolve those differences with evidence or mark them for an owner. A response that sounds more confident is not by itself a reason to choose its answer.

7.6. Hands-on exercise

You can run the OpenSpec exploration workflow against the unit-number search request. Start with the Jira story, inspect the relevant repository conventions, and record the decisions and open questions. Repeating the exploration in VS Code and IntelliJ IDEA may expose differences worth resolving before the findings are combined into one agreed decision record.

Change artifacts can wait during this exercise. The intended output is the decision record, not an early proposal built on assumptions. Once the behavior is understood well enough to review, the next chapter can turn that record into coordinated backend and frontend proposal artifacts.

7.7. Verification checkpoint

Before leaving exploration, it is worth checking every question that could block implementation. Each one should have an answer, an owner, or an explicit deferral. Also confirm that the record distinguishes:

- behavior stated by the Jira story;
- behavior inferred from existing repository conventions; and
- behavior that still requires a business or technical decision.

If those categories are mixed together, more exploration may be needed. If they are clear, the team may have enough information to decide whether to propose the change or continue investigating.

7.8. What you learned

Exploration can reduce risk when the request names a capability but leaves its observable behavior open. It gives the team a place to resolve cross-repository decisions before those decisions are copied into API code, frontend state, and tests.

A well-understood change may move directly to proposal. An unclear change may need more exploration rather than a more elaborate implementation prompt. The next artifact should record agreed behavior and remaining boundaries instead of hiding uncertainty behind confident language.

7.9. Online references

- OpenSpec skills reference
- GitHub Copilot prompt engineering

8. Proposing the First Full-Stack Change

The exploration record from the previous chapter gives the unit-number search its intended meaning. This chapter uses those decisions to shape artifacts that another developer could review and apply. The backend and frontend remain separate changes, while sharing one behavioral contract.

8.1. Learning objective

By the end of this chapter, you should be able to create and review proposal, specification, design, and task artifacts for the first truck-search capability. You should also be able to tell whether the two repository changes are ready to apply or still depend on an unresolved decision.

8.2. How proposal supports the objective

A proposal can explain why the change exists and where it stops. A specification can state what a user or client can observe. A design can explain how the repository might implement that behavior. Tasks can then make the work executable in dependency order.

OpenSpec keeps these answers in separate artifacts because they serve different readers and can change at different rates. The proposal gives reviewers a view of scope. The delta specification gives the team testable behavior. The design records implementation choices. The task list gives the agent and developer a sequence to follow.

An unresolved question from the exploration record would not belong in a requirement. It could be resolved with its owner or kept visible as a proposal dependency. A confident sentence in the wrong artifact would still describe an unresolved decision.

8.3. Start with two linked changes

One workable approach is to create an active change in each repository. The backend change would own the API contract, while the frontend change would consume it through the generated TypeScript client. Each repository would therefore have its own proposal, delta specification, design, and tasks, while the two changes shared terms and ordering constraints.

Before writing the artifacts, it can help to record the boundary between the repositories:

Decision	Backend responsibility	Frontend responsibility
Search behavior	Define the accepted input, match behavior, result shape, and API errors	Present the same behavior without redefining it

Decision	Backend responsibility	Frontend responsibility
Contract	Publish the reviewed OpenAPI document	Generate and consume the client from that document
Validation	Enforce the API boundary and describe it in OpenAPI	Give immediate feedback where the agreed behavior allows it
States	Return the agreed result, empty, and error responses	Render loading, result, empty, and error states

The decision record from chapter 7 can supply the entries in this table. The table is not meant to create new behavior. Its purpose is to expose ownership before the two proposals begin to drift.

8.4. Backend proposal

The backend proposal could connect the business request to the repository change. It would state that the capability adds unit-number search, name the affected service boundary, and list related search capabilities that remain out of scope. The rationale can stay short because the requirements and scenarios will carry the behavior.

The backend delta specification could define the observable contract selected during exploration. For each requirement, it would include scenarios for the agreed valid input, invalid input, no match, and any other case that the decision record marks as necessary. Each scenario should describe the request and observable response without prescribing a controller class or database query.

The design can then record the implementation boundary, perhaps including:

- REST endpoint and request mapping;
- validation location and error model;
- service and persistence boundaries;
- OpenAPI publication;
- tests that demonstrate the specified scenarios.

The design need not become a second requirements document. If a design choice changes observable behavior, it should return to the specification for review rather than remain hidden in implementation prose.

8.5. Frontend proposal

The frontend proposal could name the user-facing search capability and its scope. It would refer to the backend-owned contract rather than describe a second version of the API. The frontend delta specification should describe what the user can enter, what appears while the request is running, and what the user sees for a result, an empty response, or an error.

The frontend design should record the Angular boundaries that implement those scenarios:

- form and input validation;
- generated-client integration;
- loading, result, empty, and error state handling;
- component or service boundaries;
- tests for the specified interaction.

Client generation would follow the availability of the backend contract. The frontend proposal can describe that dependency, but it should avoid inventing request types or duplicating the OpenAPI contract by hand.

8.6. Order the tasks

Tasks work best when a developer can tell what must happen first and what evidence closes each task. A typical dependency order might be:

1. Record the backend proposal and delta specification from the agreed decisions.
2. Record the backend design, including the contract and test boundaries.
3. Implement and test the backend behavior, then publish the OpenAPI document.
4. Import the reviewed contract into the frontend repository and generate the client.
5. Record and implement the frontend states and interaction.
6. Run the frontend tests and the cross-application verification path.

The active OpenSpec tasks can describe this order without claiming that code exists before the apply step. A task would be complete once its code, artifact, or verification evidence exists. If a frontend task depends on an OpenAPI detail that the backend proposal has not settled, that question would need to return to review before implementation continues.

8.7. Review the artifacts

It can help to review the two changes as one capability. Start with the exploration record, then follow each requirement into its design decision and tasks. A traceability table can make that review easier:

Requirement or scenario	Design decision	Task	Evidence expected
Agreed search behavior	API and application boundary	Backend implementation and test	Passing scenario test and contract
Agreed invalid-input behavior	Validation and error mapping	Backend and frontend validation tasks	Response and visible state
Agreed no-match behavior	Empty response and UI state	Backend response and frontend rendering tasks	Empty-state test

Rows for the remaining scenarios from the decision record can be added. A blank design or task cell would suggest that the proposal is incomplete. A task with no requirement or necessary design decision might be a candidate for removal.

During review, look for three common defects:

- duplicated rules whose wording differs between repositories;
- assumptions presented as accepted behavior;
- implementation details placed in requirements without a reason.

It is also worth checking names and meanings across the contract. A result called “empty” in the backend would normally remain an empty state in the frontend unless the agreed behavior says otherwise.

8.8. Hands-on exercise

Create and review the unit-number search changes in both repositories using VS Code and IntelliJ IDEA.

The exercise can start from the chapter 7 decision record. Create the backend change first, then the frontend change with an explicit dependency on the backend-owned contract. You might ask the agent to draft each artifact separately and inspect the result before asking it to create the next one. The review can stay focused on behavior, ownership, and task order.

After the review, compare the two IDE workflows. The commands and artifacts would ideally remain the same even if the prompt entry points differ. Any difference that affects how the agent finds repository instructions or OpenSpec skills is worth recording.

8.9. Verification checkpoint

The proposal is ready to apply when the following checks hold:

- every implementation-blocking question from exploration has an answer, owner, or explicit deferral;
- each accepted requirement has observable scenarios;
- each scenario maps to a design decision and at least one task;
- the backend owns the OpenAPI contract;
- contract publication precedes client generation;
- the frontend describes states that match the backend responses; and
- no task introduces behavior outside the reviewed scope.

If a check fails, the artifact that owns the missing information is usually the right place to revise. A missing requirement is better repaired there than through a more detailed implementation task.

8.10. What you learned

A full-stack proposal is in a good position to apply when its artifacts agree about behavior, ownership, and order. The backend produces the contract, the frontend consumes it, and both repositories keep their own implementation decisions and evidence.

The next chapter applies the backend change. Its first reference point is the reviewed proposal, specification, design, and task sequence created here, rather than the original Jira story or a previous chat session.

8.11. Online references

- OpenSpec quickstart
- OpenSpec concepts
- OpenAPI Specification

Part III.

**Part III: Applying and Evolving the
Specification**

9. Applying the Spring Boot Change

The reviewed backend proposal, specification, design, and task sequence can now guide the code change. The aim is to preserve the behavior those artifacts describe while keeping implementation choices inside the approved design boundary.

9.1. Learning objective

By the end of this chapter, you can apply the unit-number change in the Spring Boot repository and trace each backend scenario to implementation code, a meaningful assertion, and local verification output.

9.2. How apply supports the objective

The task list provides the starting point for implementation. You can follow its accepted order instead of reconstructing intent from an earlier chat session or the original request. When a task exposes a missing requirement or an implementation choice outside the reviewed design, stop and return that question to review. An unresolved decision needs an answer before it becomes code.

Update task state only when the related code and evidence agree. A changed file is not evidence that the behavior works, and a passing test is not enough if the test asserts the wrong behavior.

9.3. Backend implementation

The backend design can be applied across the boundaries it names:

- validation at the agreed API boundary;
- controller behavior and response mapping;
- service behavior for the approved unit-number capability;
- repository access through the existing persistence boundary; and
- error behavior described by the specification and design.

Where the repository design calls for them, use Spring Data JPA and H2. Keep those persistence choices in the implementation. The behavioral requirements can describe what a client sends and observes without specifying which repository interface or database query supplies the result.

Keep the change within the approved unit-number scope. If implementation reveals a related search behavior that the proposal did not cover, record it as a question or follow-up. That keeps the new decision separate from the change currently under review.

9.4. Contract ownership

Treat the backend as the owner of the API contract. Generate the OpenAPI document from the implemented API, then inspect it before handing it to the Angular repository. Check the request constraints, response types, and error responses against the reviewed specification.

The generated document is part of the evidence for the backend change. If it omits a constraint, uses the wrong response shape, or describes an error as a success response, the mismatch points to an implementation defect. The frontend repository should not have to compensate by maintaining a second version of the contract.

9.5. Backend tests

Add focused unit tests where isolation clarifies validation, service, or error behavior. Add Spring Boot integration tests for the scenarios that require the application, persistence, or HTTP boundary together.

For each specified scenario, check that the test exercises the relevant boundary and asserts an observable result. Review the assertions for accidental agreement with the implementation. A test that copies an incorrect status code or response shape from the implementation may stay green while the contract is wrong.

9.6. Hands-on exercise

Apply the backend unit-number change in the Spring Boot repository. Work from the reviewed task sequence and keep the proposal, specification, design, and task files available as you apply each step. The parallel VS Code and IntelliJ IDEA instructions let you compare the two workflows; the repository artifacts and verification criteria can remain the same even when the prompt entry points differ.

When the implementation and tests are ready, generate the OpenAPI document and inspect it against the backend scenarios. Once those checks pass, the contract is ready for the frontend chapter.

9.7. Verification checkpoint

A practical handoff checkpoint is that every scenario has all three forms of evidence:

1. implementation code at the boundary responsible for the behavior;
2. a meaningful test assertion; and
3. passing local output from the relevant test or verification command.

Also confirm that the generated OpenAPI document matches the reviewed request constraints, responses, and errors. If a scenario has no test or the contract disagrees with the specification, leave the task open and repair the owning artifact or implementation before handoff.

9.8. What you learned

Applying a checked task list means preserving the reviewed behavior while making the implementation concrete. Green tests and completed tasks provide evidence to inspect, rather than automatic proof of correctness. The backend handoff can proceed when the code, tests, and generated contract agree with the scenarios.

9.9. Online references

- [Spring Boot testing](#)
- [Spring Data JPA reference](#)
- [H2 Database documentation](#)

10. Applying the Angular Change

The backend change is now implemented and its OpenAPI document can serve as the handoff artifact. In the Angular repository, use that artifact to generate the client and implement the frontend scenarios from the reviewed change. Reconstructing the API from a chat response or a manually copied TypeScript interface would leave the two repositories with different sources of truth.

10.1. Learning objective

By the end of this chapter, you can import the reviewed OpenAPI document, generate the Angular client, implement the unit-number search states, and trace each frontend scenario to a test and a running verification path.

10.2. How the contract supports the objective

In this workflow, the backend owns the API contract while the frontend handles the interaction with that contract and the way its results appear to a user. Keeping those responsibilities separate gives the two repositories a useful boundary:

Concern	Backend repository	Angular repository
Request and response schema	Publishes the reviewed OpenAPI document	Consumes generated types and methods
API validation	Defines and enforces the accepted input	Provides immediate feedback where the specification allows it
Search result	Returns the agreed response	Renders the returned data
No match and failure	Publishes the response or error contract	Maps the response to the agreed empty or error state

The table is an ownership check, not a second specification. If the generated client disagrees with the backend scenarios, the contract boundary is the first place to inspect and the owning artifact may need repair. A frontend-only request model can hide the mismatch rather than resolve it.

10.3. Generate the client

Start from the OpenAPI document produced and inspected in the previous chapter. Recording the revision or commit that supplied it makes the client reproducible for another developer.

Before generation, inspect the frontend repository instructions and its existing generated-client configuration. Check the output directory, naming conventions, package version, and the command the repository uses for

generation. The exact command belongs to the prepared repository, so a one-off command from a different project could produce misleading output.

Run the configured OpenAPI Generator task, then inspect the diff. The generated output is expected to contain the request and response types, service methods, and transport configuration described by the document. Before writing component code, check whether:

- the unit-number constraint appears where the generator exposes it;
- the success response has the fields the frontend scenario needs;
- the no-match and error responses remain distinguishable; and
- the generated service points at the expected local backend URL for the test environment.

Keeping generated files in their configured directory and leaving them unedited gives application decisions a clear home in hand-written services, components, and tests. Regenerate when the backend contract changes, when the generator configuration changes, or when a clean checkout cannot reproduce the same client. A generated diff can provide evidence of a contract change to review rather than noise to hide.

10.4. Implement the search experience

Read the frontend delta specification before choosing Angular boundaries. The UI needs to make the specified behavior visible without duplicating the backend's implementation.

10.4.1. Input and validation

Bind the search control to the input described by the specification. Client-side validation can cover rules the frontend is allowed to know, such as the settled length or empty input behavior. The backend still provides the final check because a client-side validation rule can be bypassed or become stale after the contract changes.

Keep validation messages tied to the actual state of the control. The UI can avoid sending a request for input the specification says is invalid, and it should distinguish a valid search with no record from a backend error.

10.4.2. Request and display states

Model the states the user can observe. At minimum, the component or its view model will usually need a clear path for:

1. the initial state before a search;
2. loading while the request is in progress;
3. a matching result;
4. a valid search with no matching record; and
5. an API or network error.

Each state benefits from one visible outcome and an appropriate transition. When a second search starts, the result from the first request should not appear to belong to the second. If the frontend specification allows retrying, a failure can give the user a way to recover without a page reload.

Route the request through the generated client, preferably behind a small hand-written service or facade when that matches the repository's existing Angular design. The component can then coordinate input and presentation without knowing URL paths, serializing request bodies, or reproducing the generated response type.

10.5. Trace scenarios to implementation

Use the frontend scenarios as the review spine. A traceability table can make omissions visible before the browser test runs:

Frontend scenario	UI behavior	Evidence
Accepted unit number	Enables or submits the search according to the specification	Component interaction test and successful result assertion
Invalid input	Shows the agreed validation state and avoids an invalid request	Validation or component test
Matching record	Renders the fields selected for the user-facing result	Component test and browser assertion
No matching record	Renders the agreed empty state, not an error state	Component test and browser assertion
Backend or network failure	Renders the agreed error and recovery behavior	Service/component test and browser assertion

Add rows for any scenario in the reviewed frontend specification that is not represented here. Each row need not map to one test type; the useful question is which test observes the behavior and which implementation boundary owns it.

10.6. Frontend tests

Use the narrowest test that provides meaningful evidence for the behavior under review.

- Use Vitest for pure validation or state-transition logic when the repository keeps that logic separate from Angular rendering.
- Use Angular component tests for control interaction, generated-client calls, loading behavior, result rendering, empty states, and error states. Stub the generated client at the service boundary instead of testing generated code.
- Use Playwright for the browser-to-backend path. Start the applications on the recorded lab ports or the repository's configured test ports, then assert the user-visible result rather than an internal component field.

Review the test doubles alongside the production code. A mock that always returns a result cannot prove the empty state, and a test that asserts a method was called says little about what the user saw. For each scenario, assert the observable state and, where relevant, that invalid input did not produce an API call.

10.7. Hands-on exercise

Apply the Angular unit-number change in the prepared frontend repository.

1. Import the reviewed OpenAPI document from the backend handoff and generate the configured TypeScript Angular client.
2. Inspect the generated diff and compare its types and methods with the backend scenarios.
3. Implement the input, validation, request, loading, result, empty, and error paths using the generated client.

10. Applying the Angular Change

4. Add focused component tests for every frontend scenario, then add or update the Playwright path.
5. Repeat the workflow in VS Code and IntelliJ IDEA. Compare the prompt entry points, but keep the repository commands, artifacts, and evidence the same.

Successful client generation alone is not enough to mark the frontend tasks complete. Generation shows that the document can produce code; it does not show that the UI implements the accepted behavior.

10.8. Verification checkpoint

The Angular change can move to cross-repository verification once the following evidence is available:

- the generated client comes from the reviewed backend document;
- generated files are isolated from hand-written application code;
- the UI uses generated request and response types rather than duplicate models;
- invalid input, loading, result, no-match, and error behavior each have evidence;
- component tests assert visible behavior and meaningful request boundaries; and
- the Playwright path exercises the configured frontend and backend together.

If the generated client and the specification disagree, leave the task open while the backend contract is inspected. If the contract matches but a UI scenario is missing, the frontend implementation or test needs attention. The next chapter can then validate the two repositories together and decide whether the changes are ready to synchronize and archive.

10.9. What you learned

OpenAPI generation removes manual transport duplication, but frontend review still has a role. The backend publishes the contract, the generated client carries that contract into Angular, and the frontend makes the agreed states visible to the user. That makes evidence for the behavior more meaningful than generated files or a passing request alone.

10.10. Online references

- Angular testing guide
- OpenAPI Generator TypeScript Angular generator
- Vitest guide
- Playwright documentation

11. Verifying, Synchronizing, and Archiving

The backend and frontend changes now exist in two repositories. A passing test in either repository may not, by itself, show that the change is complete. This chapter separates the evidence gathered by validation, implementation review, synchronization, and archival.

11.1. Learning objective

By the end of this chapter, you should be able to validate the OpenSpec artifacts, compare the implementation with the accepted change, synchronize accepted deltas into the main specifications, and archive a completed change without losing its history.

11.2. Four different checks

These checks address different questions. Running one generally does not replace the others.

Check	Question it answers	Evidence it produces
Artifact validation	Are the proposal, requirements, design, and tasks structured correctly?	The OpenSpec CLI result and any reported structural errors
Implementation verification	Does the code and its test suite implement the accepted behavior?	Code review, focused tests, integration evidence, and the task status
Synchronization	Have accepted requirements moved into the main specifications?	Updated main specs and the retained change record
Archival	Is the change complete and recoverable after active work closes?	An archived change containing its reason, decisions, and evidence

Artifact validation is usually the cheapest check, so you might run it first. It can catch a malformed artifact or a missing required section. It cannot prove that the application behaves correctly, that the tests exercise the right path, or that the two repositories agree.

11.3. Validate the artifacts

Run the OpenSpec validation command configured for the repository. You will usually want to validate the backend and frontend change directories separately because each repository owns its own code, tests, and specification context.

Read the result rather than treating a successful exit code as the whole review. Check that the validator examined the expected change and that the reported paths match the artifacts you edited. If validation fails, repair the artifact structure before asking Copilot to inspect the implementation. A code review built on an incomplete change description may otherwise have no stable target.

Validation also has a useful limit: it checks the shape and rules the CLI knows about. It does not decide whether a requirement is complete, whether a design is coherent, or whether an accepted scenario works in a running application.

11.4. Verify the implementation

Compare the implementation with the proposal, requirements, design, and tasks. It helps to start with the requirements and scenarios because they describe the behavior a user or another system can observe. You can then use the design and tasks to check whether the implementation follows the agreed boundaries.

For the unit-number change, one useful inspection order follows the contract:

1. Verify the backend behavior and its API document.
2. Check that the frontend client was generated from that reviewed API document.
3. Check the frontend states against the frontend requirements.
4. Run the cross-repository path and confirm the browser-visible result.

This order can make a contract mismatch easier to locate. If the backend behavior or API document is wrong, the backend-owned artifact is the natural place to repair it. If the contract is correct but the UI is missing a scenario, the frontend implementation or its tests may need the repair. Adding an undocumented workaround to one repository can make the other appear complete without resolving the underlying gap.

Use the task list as a checklist, not as proof. A completed task means that someone marked it complete. The tests and review still need to show that the accepted behavior works and that the implementation remains coherent with the design.

11.5. Synchronize accepted changes

Synchronization moves an accepted delta into the main specifications. In OpenSpec 1.13.1, it is useful to treat sync as a specification update, not as a code-generation or test command. Before running it, check that the implementation is complete, the tests pass, and the change has been accepted by the team.

Inspect the main specifications before synchronization. Note which requirements already exist, which are new, and which accepted behavior the delta changes. Then run the repository's configured sync workflow and inspect the resulting diff. Look for the accepted behavior in the main specifications, while checking that conditions remain present and the change record remains intact.

Repeat the inspection after synchronization. Check that:

- the main specifications describe the behavior now implemented;

- unchanged requirements remain unchanged;
- the delta still records why the change was made; and
- identifiers such as the Jira issue and cross-repository change identifier remain attached.

If the diff changes more than the accepted delta, pause and review the generated or proposed update. Treat synchronization as a specification update, rather than a way to resolve ambiguity by editing the specification until it fits the code.

11.6. Archive only completed work

Archiving closes the active change after its accepted behavior has been synchronized and verified. Ideally, it leaves a future reader enough context to understand what changed, why it changed, and where the supporting evidence came from.

Before archiving the backend and frontend changes, confirm that:

- artifact validation succeeds in both repositories;
- implementation tasks are complete;
- focused tests and the cross-repository path pass;
- main specifications are coherent after synchronization; and
- the change record preserves its Jira and cross-repository identifiers.

Archive the changes using the repository's configured OpenSpec workflow. Inspect the archive after the operation. A moved directory would not be sufficient if the archive had lost the proposal, requirements, design, tasks, or the reason for the change.

You will generally want to archive the two repositories only when their completion states agree. If the backend is ready but the frontend still has an open scenario, consider keeping the active work open and recording the missing evidence. The archive is most useful when it describes completed work rather than a hopeful stopping point.

11.7. The fresh-session test

A useful check after archival is to start a new Copilot session. Provide the repository artifacts, not the previous chat transcript, and ask what the unit-number change requires. See whether the session can recover the accepted behavior from the main specifications and the archived change.

This test can expose decisions that still exist only in someone's memory. When the fresh session cannot recover a condition, identifier, or boundary, return to the specifications or archive and record it there. A remembered conversation does not belong to the repository's source of truth.

11.8. Hands-on exercise

Validate, verify, synchronize when appropriate, and archive the unit-number changes in both repositories.

1. Run artifact validation in each repository and repair any structural failure.
2. Compare the backend and frontend implementation with their accepted requirements, designs, and tasks.
3. Run the focused tests and the cross-repository verification path.

11. Verifying, Synchronizing, and Archiving

4. Inspect the main specifications, synchronize the accepted deltas, and review the resulting diff.
5. Preserve the Jira and cross-repository identifiers, then archive only after both repositories meet the completion conditions.
6. Start a fresh Copilot session and recover the accepted behavior from repository artifacts.

11.9. Verification checkpoint

The unit-number change may be ready to close when the main specifications are coherent, active tasks are complete, the tests and cross-repository path pass, and the archived artifacts preserve the reason for the change. If any of those conditions is missing, consider leaving the change active and recording the missing evidence.

11.10. What you learned

Validation checks artifact structure. Tests and review verify behavior. Synchronization records accepted behavior in the main specifications. Archiving closes the active change while preserving the decisions and identifiers needed to recover it later.

11.11. Online references

- [OpenSpec CLI reference](#)
- [OpenSpec skills reference](#)
- [OpenSpec concepts](#)

12. Adding VIN Search to a Brownfield System

Adding VIN search to an existing application creates a tempting trap: reverse-engineer every old decision before writing the new specification. A delta specification gives you a narrower starting point. It records the new VIN behavior, identifies the existing behavior that must remain intact, and gives both repositories a reviewed target for implementation.

12.1. Learning objective

By the end of this chapter, you can propose an additive VIN-search change without reconstructing a complete specification for the existing applications. You can also trace the change from the OpenSpec artifacts through the backend contract, the Angular client, and the verification evidence.

12.2. Start with the accepted behavior

The unit-number search is already accepted. Use that behavior as the boundary for the new work, not as an invitation to rewrite the entire system's history.

The delta should answer two questions: Adding VIN search to an existing application can create a tempting trap: reverse-engineering every old decision before writing the new specification. A delta specification offers a narrower starting point. It can record the new VIN behavior, identify the existing behavior that must remain intact, and give both repositories a reviewed target for implementation. This distinction controls the scope of the proposal. The delta describes the VIN capability and its required interactions. It does not restate every accepted unit-number requirement. It does, By the end of this chapter, you should be able to propose an additive VIN-search change without reconstructing a complete specification for the existing applications. You should also be able to trace the change from the OpenSpec artifacts through the backend contract, the Angular client, and the verification evidence. ## Define the search modes

The unit-number search is already accepted. That behavior can serve as the boundary for the new work, rather than an invitation to rewrite the entire system's history. decisions, but they are not the same requirement. One describes the value accepted by the search; the other describes how the user reaches that validation path. it. Do not assume that two search modes share every response, message, or validation rule simply because they appear on the same screen. Record the reuse explicitly, and state any mode-specific behavior in the delta.

The resulting requirements should make the additive boundary visible:

- VIN search accepts a full 17-character VIN. The delta should help answer two questions:
- Existing unit-number behavior remains unchanged unless the delta says otherwise. The new capability has a specific input rule: VIN search accepts a full 17-character VIN. The proposal also needs to define how a user selects unit-number or VIN mode. These are related decisions, but they are not the same requirement. One describes the value accepted by the search; the other describes how the user reaches that validation path. ## Evolve the backend contract

Implement the backend change as an additive extension. Unit-number consumers should continue to receive the behavior they already depend on while the backend adds the VIN path.

The OpenAPI document is part of this change, not an afterthought. Update it to describe the reviewed contract, then inspect the document before using it to drive the frontend client. This VIN search accepts a full 17-character VIN. Users can select unit-number or VIN search mode. Existing unit-number behavior remains unchanged unless the delta says otherwise. Shared result or error behavior is reused only where the accepted specification allows it. behavior. A passing VIN test alone cannot show that the additive change stayed additive. The regression tests are the evidence that existing consumers still have their accepted path.

12.3. Evolve the Angular client

The backend change can be implemented as an additive extension. Unit-number consumers should continue to receive the behavior they already depend on while the backend adds the VIN path. generated client should reflect the reviewed OpenAPI document rather than a manually recreated The OpenAPI document is part of this change rather than an afterthought. It should describe the reviewed contract, and the document is worth inspecting before it drives the frontend client. That inspection gives the two repositories a shared boundary: the backend owns the contract, and the frontend consumes that reviewed contract. - a control for selecting unit-number or VIN mode; and Regression tests around preserved unit-number behavior belong alongside tests for the new VIN behavior. A passing VIN test alone would not show that the additive change stayed additive. The regression tests provide evidence that existing consumers still have their accepted path. mode-specific logic. Component tests can cover the visible control and its state. End-to-end tests can cover the user path through the application and the resulting search behavior. After the backend contract is updated and inspected, the TypeScript client can be regenerated. The generated client should reflect the reviewed OpenAPI document rather than a manually recreated version of the contract. Propose, apply, verify, and archive VIN search across both repositories. Keep the order visible in The Angular application then needs two pieces of behavior:

Those boundaries are best tested at the levels that own them. Unit tests can cover validation rules and mode-specific logic. Component tests can cover the visible control and its state. End-to-end tests can cover the user path through the application and the resulting search behavior. 4. Regenerate the Angular client and implement mode-specific validation. 5. Run the backend, frontend, and end-to-end tests that cover the new and preserved behavior. For the exercise, propose, apply, verify, and archive VIN search across both repositories. The order could remain visible in the change record: At each step, compare the implementation with the relevant requirement instead of treating a At each step, compare the implementation with the relevant requirement rather than treating a completed task as proof. The two repositories should describe the same change, and neither should silently change the accepted unit-number behavior. ## Verification checkpoint

A change is generally ready to close when every ADDED requirement works and accepted unit-number behavior remains unchanged. The evidence should cover the full 17-character VIN rule, mode selection, mode-specific validation, the backend contract, and the frontend path. It should also include the regression tests for unit-number consumers.

If a test or review finds a contradiction, the change should stay open. Update the delta or the implementation at the boundary where the behavior is wrong, then repeat the focused verification. The new VIN path working by itself would not be enough reason to archive the change.

12.4. What you learned

A brownfield specification can grow one reviewed change at a time. An additive delta can limit the new behavior, preserve accepted behavior explicitly, and give the backend and Angular frontend a shared contract to verify. shared contract to verify.

12.5. Online references

- [OpenSpec concepts](#)
- [OpenAPI Specification](#)
- [Playwright test assertions](#)

Part IV.

Part IV: Change Under Pressure

13. Modifying Accepted Behavior

13.1. Learning objective

By the end of this chapter, you can replace an accepted requirement without leaving contradictory rules in the source of truth. You can also trace the replacement through the backend contract, the generated Angular client, the UI, and the tests in both repositories.

13.2. Start with the accepted requirement

The original unit-number rule allows a value with 15 characters. Because that rule is already accepted, the new request would change behavior that the repositories already describe and implement rather than add a separate feature.

You can begin by locating the accepted requirement and its scenarios. It helps to record the exact rule being replaced, then list the behavior that should remain unchanged. That list is useful because changing a length constraint can affect more than one boundary. Normalization, empty input, invalid input, matching, and the result shown for a match or no match may each have their own scenario.

The change proposal should answer these questions before implementation:

- Which accepted requirement is being replaced?
- What does the revised maximum mean at the boundary of 10 characters?
- Does normalization happen before the length check, and does the change alter that behavior?
- What happens for empty input?
- Which existing match, no-match, and error behavior remains unchanged?

If the team has not decided one of these points, it may be better to keep it open for review. An agent should not choose a rule merely because the code currently makes that choice.

13.3. Write a complete modification

OpenSpec distinguishes a modified requirement from an added requirement. An added requirement introduces behavior that did not exist in the accepted specification. A modified requirement replaces the accepted requirement with a new complete version.

For this change, the delta would replace the 15-character rule with a maximum of 10 characters. It should carry revised scenarios for the valid boundary, values beyond the boundary, empty input, normalization, and the behavior that remains unchanged. The exact scenarios depend on the decisions made during exploration, but the delta needs to contain those decisions rather than pointing back to an old rule and asking the implementer to combine the two.

13. *Modifying Accepted Behavior*

Appending a second requirement that says “maximum 10 characters” would leave both rules in the source of truth. A reader could not tell whether 15 characters is still valid, whether the newer rule has priority, or which scenarios belong to which rule. The implementation agent would face the same ambiguity. Modification semantics remove the old rule and make the revised requirement the one that survives synchronization into the main specification.

Review the delta as a set of operations:

- **ADDED** introduces a new requirement.
- **MODIFIED** replaces an accepted requirement and carries its complete revised behavior.
- **REMOVED** deletes a requirement that no longer applies.
- **RENAMED** changes the requirement’s identity while preserving the intended behavior.

The distinction that matters here is between **ADDED** and **MODIFIED**. The unit-number length change fits **MODIFIED** because the new rule replaces an accepted one.

13.4. Full-stack impact

The accepted requirement may appear in more places than the main specification. A search of both repositories for the old constraint can establish the impact surface before the change is applied. The search can include the OpenSpec artifacts, backend validation, OpenAPI constraints, generated client output, frontend validation, UI text, and tests. It does not by itself decide which files should change.

Backend validation can enforce the revised rule, while the OpenAPI contract describes the same constraint. Inspecting the generated contract before regenerating the Angular client gives the team a chance to catch a mismatch. The frontend can then consume the reviewed contract rather than preserve an older generated constraint.

Frontend validation and any UI text that still describes 15 characters would also need an update. The existing search behavior that the delta explicitly preserves can remain unchanged. A passing test for a ten-character value would not be enough if a test, error message, or generated schema still accepts or describes the old limit.

Use the tests to cover both sides of the replacement:

- the new maximum and the value beyond it;
- normalization and empty input, according to the reviewed decisions;
- preserved match, no-match, and error behavior; and
- consistent validation across the backend and Angular client.

13.5. Hands-on exercise

The exercise is to implement and archive the revised unit-number rule in both repositories. The change record can keep the sequence visible:

1. Locate the accepted 15-character requirement and its scenarios.
2. Decide the revised boundary, normalization, empty-input, and preserved-behavior rules.
3. Create a **MODIFIED** delta containing the complete replacement requirement and scenarios.
4. Update backend validation and the OpenAPI constraint.
5. Regenerate the Angular client and update frontend validation, UI text, and affected tests.

6. Verify the implementation against the delta before synchronizing and archiving it.

At each step, compare the latest artifact with the previous accepted behavior. If implementation work reveals an unanswered product decision, pause and revise the delta. Resolving the gap with a code-only change or a corrective chat message would leave the source of truth unchanged.

13.6. Verification checkpoint

Search the specifications, code, generated artifacts, UI text, and tests for obsolete 15-character assumptions. Then inspect the final source-of-truth result that archive will produce.

The change could be ready to close when:

- the accepted requirement has one complete replacement rather than two contradictory rules;
- the maximum-of-10 behavior and its boundary scenarios pass;
- normalization and empty-input behavior match the reviewed decisions;
- backend and frontend validation describe the same constraint;
- generated artifacts no longer carry the old constraint; and
- preserved search, match, no-match, and error behavior still has verification evidence.

If the search finds an obsolete assumption, the change can stay open while the affected artifact, code, generated output, or test is updated and the focused verification is repeated. Archiving would make sense only after the accepted specification and the implementation describe the same behavior.

13.7. What you learned

A modified delta works as a replacement rather than an additional opinion about the same behavior. Carrying the complete revised requirement and its scenarios helps prevent the old and new unit-number rules from coexisting accidentally. Searching the repositories and generated outputs then checks whether the replacement reached every boundary where the accepted rule was used.

13.8. Online references

- OpenSpec documentation
- Jakarta Bean Validation
- Angular forms guide

14. Revising an In-Flight Change

14.1. Learning objective

By the end of this chapter, you can revise an active change when the business changes its mind before implementation is complete. You can also decide whether the revision belongs in the existing change or in a separate one, then trace the decision through the specifications, API, UI, generated client, and tests.

14.2. Start with the decision that changed

An active change can be treated as a working agreement rather than a permanent commitment. The team may discover a missing business rule, reject an earlier assumption, or widen the requested behavior during implementation. When that happens, the next step is usually to update the artifact that records the decision. A corrective message in chat may reach the current implementer, but it does not update the instructions that reviewers, later agents, or the next developer will read.

The interrupted proposal in this chapter began with one filter:

- filter vehicles by Fleet Origin State;
- add Active Status values of Active and Decommissioned; and
- decide how the filters combine and how empty selections behave.

The new request changes the shape of that work. It is no longer enough to implement the original filter and append a second control. The proposal needs to describe the combined behavior that the user now expects.

Before changing the artifact, write down the decision that moved. Ask:

- Is Active Status an additional filter, or does it replace part of the original proposal?
- Do the filters combine with AND, OR, or a rule that depends on the selected values?
- What does an empty Fleet Origin State selection mean?
- What does an empty Active Status selection mean?
- Which completed work still supports the revised behavior?

It helps to keep unanswered questions visible during review. Reviewers should not have to infer product behavior from whatever the current query or UI happens to do.

14.3. Rebuild the active artifact

Once the decision is clear, revise the active change as a complete description of the new behavior. Revisit its scope, affected capabilities, requirements, scenarios, and task list. Stale text is usually better replaced than left beside a second statement for the reader to reconcile.

The revision will usually need to make these relationships explicit:

14. *Revising an In-Flight Change*

- the filter controls available to the user;
- the values each control accepts;
- how multiple selections combine;
- what the API receives when a control has no selection; and
- what results and empty states the UI presents.

Then trace the effects through the implementation. A change to filter behavior can affect API parameters, query construction, validation, generated-client types, UI controls, and tests. The exact files depend on the two repositories, but the artifact should name the affected surfaces before implementation resumes.

It is also worth revisiting tasks in the same pass. A completed task remains complete only when its result still satisfies the revised requirement. A task that implemented the one-filter query may need to be reopened if the new combination rule invalidates that query. A task that added shared plumbing may remain complete if the revised scenarios still exercise it.

This distinction helps avoid two common errors. Marking every earlier task incomplete throws away work that the new design still uses. Leaving every task complete hides work that no longer proves the requested behavior.

14.4. **Decide whether to amend or split the change**

Not every new request belongs in the active proposal. Four questions can help with the decision:

- How much implementation is already complete?
- Does the new request change the same behavior or introduce an independent capability?
- Does the release still treat both requests as one outcome?
- Would a reviewer understand the final proposal without reconstructing its history?

The active change is usually the better home when the new decision corrects or completes the behavior already under review. The filter combination rule belongs with the filter proposal because it defines what that proposal means.

A separate change may be appropriate when the new request has an independent purpose, review path, or release decision. A separately valuable change is usually better kept outside the active proposal, even when the code touches the same screen. Shared files create coordination work, but they do not make two business decisions one change.

The change record can make that choice visible. A short note explaining why the proposal was amended or split gives reviewers a way to check the scope without relying on chat history.

14.5. **Keep implementation and review synchronized**

After revising the artifact, review it before asking an agent to continue. The review can check whether the requirements and scenarios describe the same combination rules, whether the API contract can represent those rules, and whether the UI tasks still describe the original one-filter design.

Then compare existing work with the revised scenarios. For each completed item, ask what evidence it provides. Code that still passes a narrow unit test may not satisfy the revised behavior if the test never combines the filters. A generated client may compile while still exposing an obsolete parameter shape. A label or help message may describe only the first filter even after the query supports both.

When the review finds a gap, leave the proposal open. Revise the requirement or scenario first, then update the affected task and implementation. This order keeps the source of truth ahead of the code and gives the next verification pass a precise target.

14.6. Hands-on exercise

Revise, review, apply, and verify the active filter change using the documented OpenSpec 1.13.1 workflows in both IDEs.

1. Locate the active change and record the original one-filter behavior.
2. Add the Active Status decision and resolve the filter-combination and empty-selection questions.
3. Revise the requirements and scenarios so they describe the complete combined behavior.
4. Reassess the API parameters, query design, generated client, UI controls, and affected tests.
5. Reopen or reorder tasks that the new decision invalidated, while preserving work that still provides evidence.
6. Review and apply the revised change, then verify the implementation before closing it.

At each step, compare the active artifact with the code and tests. If an implementation detail exposes an unanswered product decision, return to the artifact and record the decision there. A code-only patch or a chat message that disappears from the change history would leave that gap unresolved.

14.7. Verification checkpoint

Search the active artifacts, specifications, tasks, code, generated output, tests, and UI text for assumptions from the original one-filter design. Then verify the revised scenarios, including the cases where one or both filters have no selection and where both filters are applied together.

You can consider closing the change when:

- the active artifact states one coherent filter behavior;
- the combination and empty-selection rules are explicit;
- the API, query, generated client, and UI use the same behavior;
- stale tasks and prose no longer describe only the original filter; and
- tests provide evidence for the revised scenarios and the behavior that remains unchanged.

If the search finds an obsolete assumption, leave the change open. Update the artifact or implementation that owns the assumption, then repeat the focused verification. The change can be archived after the accepted specification and the implementation describe the same behavior.

14.8. What you learned

An in-flight change works best when its source of truth is updated before the work that depends on it is re-evaluated. The right unit of review is the behavior, not the number of files already changed. Completed work can remain in place when it still proves the revised scenarios, while work whose evidence the new decision invalidates may need to be reopened.

14.9. Online references

- OpenSpec skills reference
- OpenSpec CLI reference

15. Coordinating Parallel Changes

15.1. Learning objective

By the end of this chapter, you should be able to manage concurrent changes without treating isolated folders or branches as proof that the changes are compatible. You should have a way to compare active deltas with the same accepted baseline, expose semantic overlap, and choose an archive order that can preserve both business decisions.

15.2. Isolation is not compatibility

Parallel work tends to create two separate coordination problems. Git can isolate edits to files, and OpenSpec can isolate proposed intent in separate change folders. Neither mechanism, by itself, shows that the changes can be accepted together.

Two changes can pass their own reviews and tests while making incompatible decisions about the same behavior. They may add different values to one search contract, define different meanings for a shared request parameter, or ask the frontend to expose two controls that the backend cannot represent together. The conflict is semantic even when the files do not conflict textually.

It helps to keep the mechanisms distinct:

- An OpenSpec change folder isolates a proposal, its requirements, design, and tasks from other proposals.
- A Git branch isolates a line of commits from another line of commits.
- A worktree gives a branch its own working directory so developers or agents can work in parallel.

These mechanisms can reduce accidental edits. They do not, on their own, answer whether the proposals describe one coherent product behavior. Comparing each proposal with the accepted specification and with the other active changes can answer that question more directly.

15.3. A pair of truck-search changes

Consider two active changes in the truck-search system:

- one adds License Plate search; and
- another modifies VIN-related behavior.

The changes have different business goals, so separate proposals make sense at first. Each proposal should describe its own input rules, result behavior, and non-goals. The separation lets reviewers discuss one decision without mixing it with the other.

The proposals would still benefit from a shared-surface review. Compare them against the latest accepted specifications and inspect whether both changes affect:

15. Coordinating Parallel Changes

- the backend search contract;
- the generated Angular client;
- the control that selects or submits a search mode;
- shared validation and error behavior; and
- regression tests for the search modes that already work.

The purpose of this comparison need not be to reject parallel work. It can instead surface decisions that may need to be made once. For example, if both proposals change the shared search request, the team may need to agree on a request shape and mode-selection rule before either change becomes the new baseline. A frontend proposal that invents a client shape independently of the backend proposal may already be behind the source of truth.

15.4. Compare deltas before archive

One useful way to review each active change is twice: first against the accepted baseline, then against the other active changes. The first comparison asks what the change adds, modifies, or preserves. The second asks whether another proposal adds, modifies, or removes the same behavior.

Start with the behavioral requirements, not the file list. For each requirement, record:

- the accepted behavior it assumes;
- the behavior it changes or adds;
- the shared contract or UI surface it touches; and
- the evidence that will show the behavior works.

Then compare the records. Look for more than identical words. These are overlap signals:

- both changes alter the same endpoint or generated client method;
- both changes define a different validation rule for the same input;
- both changes change how the UI selects a search mode;
- one change assumes a result shape that the other changes; or
- a regression test in one change would fail after the other change is synchronized.

When a comparison finds overlap, return to the requirements and scenarios. Resolving the disagreement through implementation order alone could make the behavior accidental. The team could combine the changes, revise one proposal, or keep both with an explicit compatibility decision. The choice belongs in the active artifacts so a reviewer can recover it without the originating chat session.

15.5. Coordinate the contract boundary

The backend owns the OpenAPI contract in this book's two-repository lab. That ownership suggests a practical order for the parallel changes: agree on the backend behavior, review the resulting contract, then regenerate the Angular client before completing frontend work.

This order does not mean that frontend work has to wait for every backend task. A frontend developer can prepare tests or UI structure against a reviewed proposal. The generated client would still need to come from the accepted backend contract. A hand-written client type may hide an unresolved contract decision rather than solve it.

Keep the following questions visible while the changes are active:

- Can the contract represent License Plate and VIN searches without ambiguous combinations?
- Which input rules belong to each mode, and where are they enforced?
- Does the shared UI expose the modes consistently with the contract?
- Which existing unit-number and VIN scenarios must remain unchanged?
- Which generated files and frontend tests must be regenerated or revised after synchronization?

An answer that applies to both proposals belongs in the shared specification or in an explicit coordination decision. An answer that applies to only one capability belongs in that change's requirements. Keeping those scopes separate helps prevent a local assumption from becoming an accidental rule for every search mode.

15.6. Reconcile and archive

Archive order can matter because synchronization changes the accepted baseline. The first archived change becomes the comparison point for the next one. In many cases, choosing the order from dependency makes more sense than choosing it from which developer finished first.

When the other change depends on a shared contract or prerequisite behavior, it usually makes sense to archive the change that establishes it first. If the changes are independent at the contract level, either order may work, but the second change still needs a fresh comparison against the updated baseline.

After the first change is synchronized and archived:

1. Re-read the second change against the latest main specifications.
2. Replace stale assumptions in its requirements, design, and tasks.
3. Recheck its API and generated-client impact against the accepted contract.
4. Re-run the overlap review and record any decision that changed.
5. Apply, verify, synchronize, and archive only after the revised change is coherent.

Avoid silently rewriting the second change to fit the first. If the first change changed a shared decision, update the second artifact so the new behavior remains reviewable. If the goals are no longer compatible, keep the second change open and ask for a product decision rather than preserving two contradictory rules in the main specification.

15.7. Hands-on exercise

Create two active changes for the prepared truck-search repositories: one for License Plate search and one for a VIN-related behavior change.

1. Write the business goal, affected behavior, non-goals, and scenarios for each change.
2. Inspect both changes against the same accepted specifications.
3. Mark every shared backend contract, generated-client, UI, and regression-test surface.
4. Identify one semantic overlap and record the decision that resolves it.
5. Choose an archive order based on the dependency between the two changes.
6. Synchronize the first change, then revise the second against the new baseline.
7. Apply and verify the second change without removing either business goal.

At each step, compare the artifact with the implementation and tests. If a conflict appears only after the first change is archived, that is a sign that the second change may have become stale. A code patch that leaves the active artifact describing the old contract will make the discrepancy harder to review.

15.8. Verification checkpoint

One useful checkpoint is to validate the active artifacts before implementation, then validate the final main specifications after synchronization. Run the regression tests for existing search modes as well as the tests for the two reconciled changes. Include contract generation and frontend tests when the shared API or client changed.

The work is ready to close when the evidence supports all of the following:

- each active change has a distinct business purpose and an explicit scope;
- shared requirements have one agreed interpretation;
- the accepted specifications contain no contradictory search rules;
- the generated client reflects the reviewed backend contract;
- the UI and validation match the selected search behavior; and
- affected new and regression scenarios have evidence from tests.

If the second change still refers to the pre-archive baseline, keep it active. Update the stale artifact, implementation, or test, then repeat the focused verification. Isolation makes the work possible; comparison and synchronization are what can make the result coherent.

15.9. What you learned

Parallel changes are independent proposals, but they do not necessarily become independent realities. Git branches and worktrees protect concurrent edits, while OpenSpec records the decisions each change intends to make. The team still needs to compare those decisions against one accepted baseline, coordinate shared contracts, and re-review any change that becomes stale after synchronization.

15.10. Online references

- [OpenSpec documentation](#)
- [Git worktree documentation](#)

16. Making SDD a Team Practice

16.1. Learning objective

Establish a proportionate, reviewable OpenSpec workflow that can survive beyond the book's exercises.

The exercises used one person, two repositories, and a controlled sequence of changes. A team has to answer harder questions: who owns a decision, how much specification work a change deserves, and how a reviewer can recover the reasoning after the original Copilot session is gone. SDD becomes a team practice when the repository records those decisions without turning every edit into a formal project.

16.2. Put decisions in the right place

An agent can propose code, tests, and artifact text. It cannot decide whether a business behavior is acceptable, whether a contract change is compatible with existing clients, or whether a risk is worth the cost. A human owner must make those decisions and leave enough evidence for another person to review them.

Use the repository artifacts for engineering intent and Jira for work tracking:

- Jira records the work item, status, priority, and responsible people.
- `proposal.md` records the business goal, scope, and non-goals for a change.
- Delta specifications record the behavior the team will add, modify, or remove.
- `design.md` records implementation decisions and their constraints.
- `tasks.md` records the work sequence and its verification points.
- Commits, pull requests, and test output provide evidence that the accepted intent was implemented.

This division prevents two common failures. A Jira ticket should not be the only place where an agent can find a behavioral rule, and an OpenSpec change should not become a second work-tracking system.

16.3. Make traceability recoverable

Traceability is useful when it lets a reviewer move from a decision to its evidence without asking the original author to reconstruct the conversation. Define a small naming and linking convention for both repositories. For example, a Jira issue can link to the OpenSpec change name, the pull request can mention the same issue, and the verification record can point to the relevant test command or result.

The exact convention matters less than using it consistently. At minimum, a reviewer should be able to answer these questions:

- Which business decision does this change implement?
- Which requirements and scenarios express that decision?
- Which code, generated artifacts, and tests carry it out?
- What evidence supports synchronization and archive readiness?

Keep the links close to the artifacts they explain. A change that affects the backend contract and the Angular client should identify both repositories and the point at which the client must be regenerated. That makes the contract boundary visible during review instead of leaving it implicit in task order.

16.4. Assign review ownership

Review ownership follows the decision being reviewed. The same person may hold several roles on a small change, but the roles should still be explicit.

- A product or business owner reviews behavior, scope, and non-goals when the change affects what the system promises.
- A technical reviewer checks contracts, architecture, data handling, and integration boundaries.
- An implementation reviewer checks that code and tests satisfy the reviewed requirements.
- The person responsible for synchronization or archival checks that the accepted specifications and evidence are complete.

These reviews do not require four people or four meetings. They require the team to name the questions that must be answered before work moves forward. A behavior change needs business review. An OpenAPI or persistence change needs technical review. A routine, reversible edit may need only a short recorded decision and focused verification.

16.5. Scale the artifacts to the risk

OpenSpec is a control for uncertainty and coordination. It is not a requirement that every line change receive the same amount of ceremony. Before creating a full change, classify the work by its uncertainty, reversibility, and blast radius.

Use a lighter record for a routine, low-risk, reversible change whose behavior is already covered by the accepted specifications. Use a full proposal, delta specification, design, task list, and verification record when the change crosses a repository boundary, changes a contract, modifies accepted behavior, or creates a meaningful regression risk. Urgency does not remove the need to record a decision. It may justify a smaller initial artifact and a follow-up review.

Record deliberate exceptions. A short note should state why the full workflow was unnecessary and what evidence still supports the change. This gives the next reviewer a reason for the missing artifacts instead of making the repository look incomplete by accident.

16.6. Maintain the context that agents use

The workflow itself changes over time. OpenSpec versions, Copilot integration files, generated clients, commands, and test tooling can all make an old instruction inaccurate. Treat those files as maintained engineering assets.

Review the repository context when you upgrade OpenSpec or the Copilot integration. Remove duplicated instructions when two files begin to disagree. Check command examples against the versions used by the repositories. Revisit generated integration files after an update rather than assuming that a new version preserves the old layout or behavior.

The same rule applies to the book's examples. A command that worked for the prepared repositories at one version can become misleading after a tool upgrade. Keep version-sensitive guidance tied to a tested version, and make the version visible when a reader needs it to reproduce the result.

16.7. Capstone: defend the amount of specification

Complete the capstone by proposing a new commercial truck search change across both repositories. The purpose is not to produce the largest possible set of artifacts. It is to defend the amount of specification work the change needs.

1. State the business goal, affected behavior, non-goals, and owner.
2. Classify the change by uncertainty, reversibility, urgency, and blast radius.
3. Choose the OpenSpec artifacts that follow from that classification.
4. Record the backend contract boundary and the frontend impact, including generated-client work.
5. Assign business and technical review where the change requires those decisions.
6. Link the Jira issue, OpenSpec change, commits, pull requests, and verification evidence.
7. Apply, verify, synchronize, and archive the change only after the accepted specifications and implementation agree.

16.8. Verification checkpoint

Ask a reviewer in a fresh session to reconstruct the capstone's intent and evidence using repository artifacts alone. The reviewer should be able to identify the business decision, the changed behavior, the responsible reviewers, the contract boundary, and the evidence for completion without opening the originating chat session.

If the reviewer cannot do that, find the missing decision or link before treating the change as a team practice success. The problem may be an underspecified requirement, an unrecorded exception, a stale instruction, or a verification step that produced no recoverable evidence.

16.9. What you learned

SDD works as an engineering control when the team records intent, assigns human ownership, scales the artifacts to the risk, and preserves evidence through synchronization and archive. It also has a cost: someone must review the decisions, maintain the context, and keep the artifacts current. A proportionate workflow makes that cost visible instead of hiding it in repeated rework or fragile agent sessions.

16.10. Online references

- OpenSpec documentation
- GitHub Copilot customization
- Atlassian Jira documentation

17. Summary

OpenSpec gives an AI-assisted team a durable place to keep intent, but the files alone do not create alignment. The team must still ask precise questions, review generated artifacts, test the implementation, and reconcile overlapping decisions.

The truck-search sequence demonstrates the core patterns:

- A greenfield change establishes the first accepted behavior through explore, propose, apply, and archive.
- An additive delta introduces VIN search without replacing unit-number search.
- A modified delta replaces the old unit-number constraint with a maximum of 10 characters.
- An active proposal can be revised before implementation when the business changes direction.
- Parallel changes require both artifact isolation and source-control isolation, followed by deliberate synchronization.

The practical test is whether a developer or agent can enter the repository in a later session and recover what the system should do, what is changing, and how completion will be verified. When the specs, code, and tests answer those questions consistently, context has become shared engineering material rather than chat history.

References

The links below are the primary sources used for current commands, repository behavior, and technology details. Version-sensitive claims should be checked against the source at the time a team applies them.

OpenSpec and workflow

- OpenSpec repository¹
- OpenSpec documentation²
- OPSX workflow³
- Installation⁴
- Supported tools and aliases⁵
- Awesome-OpenSpec⁶

Petclinic and technology

- Spring Petclinic Angular⁷
- Spring Petclinic REST⁸
- Spring Boot reference⁹
- Spring Framework reference¹⁰
- Angular documentation¹¹
- Java SE 21 documentation¹²
- TypeScript documentation¹³

AI-assisted development

- GitHub Copilot in VS Code¹⁴

¹<https://github.com/Fission-AI/OpenSpec>

²<https://github.com/Fission-AI/OpenSpec/tree/main/docs>

³<https://github.com/Fission-AI/OpenSpec/blob/main/docs/opsx.md>

⁴<https://github.com/Fission-AI/OpenSpec/blob/main/docs/installation.md>

⁵<https://github.com/Fission-AI/OpenSpec/blob/main/docs/supported-tools.md>

⁶<https://github.com/speclib/awesome-openspec>

⁷<https://github.com/spring-petclinic/spring-petclinic-angular>

⁸<https://github.com/spring-petclinic/spring-petclinic-rest>

⁹<https://docs.spring.io/spring-boot/reference/>

¹⁰<https://docs.spring.io/spring-framework/reference/>

¹¹<https://angular.dev/>

¹²<https://docs.oracle.com/en/java/javase/21/>

¹³<https://www.typescriptlang.org/docs/>

¹⁴<https://code.visualstudio.com/docs/copilot/overview>

References

- GitHub Copilot CLI¹⁵

¹⁵<https://docs.github.com/en/copilot/concepts/agents/about-copilot-cli>

A. Installing and Updating OpenSpec

A.1. Scope

This appendix is intended to cover tested OpenSpec 1.13.1 installation and maintenance procedures for Windows 11 and macOS. The operating-system differences should remain visible rather than being folded into one generic command sequence.

A.2. Before installation

- Readers should first confirm that Node.js 24.19.0 and npm are available.
- The appendix should describe global npm package locations and the way PATH affects command discovery.
- Teams may want to decide whether to pin OpenSpec 1.13.1 or allow controlled upgrades.

A.3. Install OpenSpec 1.13.1

- Windows 11 and macOS should have separate terminal instructions.
- After installation, readers can verify the executable and its version.
- The tested baseline should include the expected command output.

A.4. Inspect the installed version

- The appendix can show the OpenSpec version command.
- It can also show how npm reports local and global package versions.
- The CLI package version should be distinguished from repository-generated integration files.

A.5. Upgrade OpenSpec

- Before changing the team baseline, readers should check the release notes.
- The global npm package can then be upgraded.
- The new version should be verified before any repositories are changed.

A.6. Update an existing repository

- An existing repository can be updated with `openspec\update`, using the intended profile and GitHub Copilot integration.
- Before committing, readers should review changed instructions, prompt files, and Agent Skills.
- Slash-command discovery should be retested in VS Code and IntelliJ IDEA.
- The appendix should explain when `-V-force` may be appropriate and what needs review afterward.

A.7. Verification checklist

- From a new terminal, the expected CLI version should run.
- Both repositories should report healthy OpenSpec roots.
- Generated GitHub Copilot files should match the selected profile.
- Existing project-owned configuration and artifacts should remain intact.

A.8. Online references

- OpenSpec installation
- OpenSpec CLI reference
- OpenSpec supported tools
- npm documentation

B. OpenSpec CLI Command Reference

This appendix offers a quick reference for the openspec CLI commands used throughout the book. OpenSpec slash commands are covered in Appendix B. The CLI command names and examples are current as of OpenSpec CLI 1.13.1. Since your installed version may differ, you may want to run `openspec\ -V-help` or `openspec\TU\textless{ }command\TU\textgreater{ } -V-help` to check the syntax available to you.

The CLI helps manage the files and relationships that make a change reviewable. Decisions about a change's scope and the quality of its requirements remain with the people reviewing the work.

B.1. Typical workflow

1. You might run `openspec\init` once in a project, then use `openspec\update` when the generated instruction files need to catch up.
2. To begin a change, use `openspec\new\change\TU\textless{ }name\TU\textgreater{ }`, then inspect its artifact status with `openspec\status -V-change\TU\textless{ }name\TU\textgreater{ }`.
3. You can use `openspec\list`, `openspec\show`, `openspec\context`, and `openspec\instructions` to inspect the repository and work out what an agent needs next.
4. Before treating the change as ready, consider running `openspec\validate\TU\textless{ }name\TU\textgreater{ }`. In CI, `openspec\validate -V-all` may be a better fit when the repository is the unit of validation.
5. After implementation and verification, you can run `openspec\archive\TU\textless{ }name\TU\textgreater{ }`. Archiving updates the main specifications and moves the completed change out of the active work area.

OpenSpec reports failures through the process exit status, which makes commands such as `validate` suitable for scripts and CI. The CLI help does not promise a numbered list of exit codes, so this reference does not invent one. In a pipeline, treat a nonzero status as a failed check and preserve the command output as the diagnostic.

B.2. Core commands

The commands below cover the operations most projects use regularly. The `-V-json` options can help when another tool needs to consume a result, but reading the Markdown artifacts still matters during review.

Command	Purpose	Useful options
<code>openspec\init\{ }path{ }</code>	Initialize OpenSpec in a project and write the selected tool instruction files.	<code>-V-tools\all</code> , <code>-V-tools\github-copilot</code> , <code>-V-language\TU\textless{ }language\TU\textgreater{ }</code> , <code>-V-profile\TU\textless{ }profile\TU\textgreater{ }</code> , <code>-V-force</code>

Command	Purpose	Useful options
<code>openspec\update\{[]path{[]}</code>	Update OpenSpec instruction files after the CLI or tool configuration changes.	<code>-V-force</code>
<code>openspec\new\change\TU\textless{ }name\TU\textgreater{ }</code>	Create a new change directory using the selected workflow schema.	<code>-V-description\TU\textless{ }text\TU\textgreater{ },</code> <code>-V-goal\TU\textless{ }text\TU\textgreater{ },</code> <code>-V-schema\TU\textless{ }name\TU\textgreater{ },</code> <code>-V-store\TU\textless{ }id\TU\textgreater{ }, -V-json</code>
<code>openspec\list</code>	List active changes.	<code>-V-changes, -V-specs,</code> <code>-V-sort\recent\TU\textbar{ }name, -V-json,</code> <code>-V-store\TU\textless{ }id\TU\textgreater{ }</code>
<code>openspec\show\{[]item-name{[]}</code>	Display a change or specification.	<code>-V-type\change\TU\textbar{ }spec, -V-json, -V-diff,</code> <code>-V-deltas-only,</code> <code>-V-store\TU\textless{ }id\TU\textgreater{ }</code>
<code>openspec\status\ -V-change\TU\textless{ }id\TU\textgreater{ }</code>	Show which artifacts are complete for a change.	<code>-V-all,</code> <code>-V-schema\TU\textless{ }name\TU\textgreater{ },</code> <code>-V-json, -V-store\TU\textless{ }id\TU\textgreater{ }</code>
<code>openspec\validate\{[]item-name{[]}</code>	Validate a change or specification.	<code>-V-all, -V-changes, -V-specs, -V-strict, -V-json,</code> <code>-V-type\change\TU\textbar{ }spec,</code> <code>-V-store\TU\textless{ }id\TU\textgreater{ }</code>
<code>openspec\archive\{[]change-name{[]}</code>	Archive a completed change and update the main specifications.	<code>-V-yes, -V-skip-specs, -V-no-validate, -V-json,</code> <code>-V-store\TU\textless{ }id\TU\textgreater{ }</code>
<code>openspec\doctor</code>	Report relationship health for the resolved OpenSpec root.	<code>-V-json, -V-store\TU\textless{ }id\TU\textgreater{ }</code>

The command names reflect two related interfaces. `openspec\new\change` creates a change. The older grouped forms, `openspec\change\show`, `openspec\change\list`, and `openspec\change\validate`, can inspect and validate changes. Since the help output marks `openspec\change\list` as deprecated, new scripts are better served by the top-level `list`, `show`, and `validate` commands.

For a quick inspection of a repository, you could start with:

```
openspec list
openspec status --all
openspec doctor
```

For a CI (continuous integration) check that covers both active and archived material, choose the narrowest validation that matches the pipeline's purpose. `openspec\validate\ -V-all` validates all changes and specifications, while `openspec\validate\ -V-archived` checks that archived changes have completed tasks.

B.3. Store and workset commands

Stores and worksets address different needs. You can think of a store as a standalone OpenSpec repository registered on the local machine, giving several related projects a shared place for specifications and changes.

A workset is a personal, local view that composes folders for a working session. Removing a workset does not remove its member folders.

Command	Purpose	Useful options
<code>openspec\store\setup\{[]id{}}</code>	Create and register a local store.	<code>-V-path\TU\textless{ }path\TU\textgreater{ }</code> , <code>-V-remote\TU\textless{ }url\TU\textgreater{ }</code> , <code>-V-init-git</code> , <code>-V-no-init-git</code> , <code>-V-json</code>
<code>openspec\store\register\{[]path{}}</code>	Register an existing local store.	<code>-V-id\TU\textless{ }id\TU\textgreater{ }</code> , <code>-V-yes</code> , <code>-V-json</code>
<code>openspec\store\list</code>	List stores registered on this machine.	<code>-V-json</code>
<code>openspec\store\doctor\{[]id{}}</code>	Check a store's registration and metadata.	<code>-V-json</code>
<code>openspec\store\unregister\TU\textless{ }id\TU\textgreater{ }</code>	Forget a registration without deleting the files.	<code>-V-json</code>
<code>openspec\store\remove\TU\textless{ }id\TU\textgreater{ }</code>	Forget a registration and delete its local folder.	<code>-V-yes</code> , <code>-V-json</code>
<code>openspec\context</code>	Print the working context for the resolved OpenSpec root.	<code>-V-json</code> , <code>-V-store\TU\textless{ }id\TU\textgreater{ }</code> , <code>-V-code-workspace\TU\textless{ }path\TU\textgreater{ }</code> , <code>-V-force</code>
<code>openspec\workset\create\{[]name{}}</code>	Save a named view of project folders.	Repeat <code>-V-member\TU\textless{ }path\TU\textgreater{ }</code> ; optionally use <code>-V-tool\TU\textless{ }id\TU\textgreater{ }</code> and <code>-V-json</code>
<code>openspec\workset\list</code>	Show saved worksets and their members.	<code>-V-json</code>
<code>openspec\workset\open\TU\textless{ }name\TU\textgreater{ }</code>	Open a saved workset in its preferred tool.	<code>-V-json</code>
<code>openspec\workset\remove\TU\textless{ }name\TU\textgreater{ }</code>	Delete a saved workset without touching member folders.	<code>-V-json</code>

When the working root should come from a registered store rather than the current project, you can add `-V-store\TU\textless{ }id\TU\textgreater{ }` to commands that support it. If shared context is missing or unexpectedly empty, it may help to run `openspec\store\doctor` first. That checks the registration boundary before you inspect individual artifacts.

OpenSpec Stores are still in BETA

Please keep in mind the OpenSpec Stores feature is still in BETA release and may change before they are released as a production feature in OpenSpec.

C. GitHub Copilot Commands and OpenSpec Skills

OpenSpec names each workflow by its Agent Skill, while the command a developer types depends on the agent harness running it. This appendix covers GitHub Copilot inside VS Code, IntelliJ IDEA, and GitHub Copilot CLI. VS Code and IntelliJ IDEA use GitHub Copilot's prompt-file integration. Because GitHub Copilot CLI does not read `.github/prompts/`, the installed skills are the appropriate option there.¹

The appropriate integration can install the relevant files when you initialize the repository, for example with `openspec\init\ -V-tools\github-copilot`.² The table uses the current OPSX command form for the Copilot extensions and the Agent Skill form for Copilot CLI. OpenSpec documents the command pattern using the apply workflow; the other workflows appear to follow the same host-specific naming pattern.³

OpenSpec action	When to use it	VS Code	IntelliJ IDEA	GitHub Copilot CLI
explore	Investigate an idea, existing behavior, or an unclear requirement before creating a change proposal. It is a thinking step and does not implement code.	<code>/opsx-explore\\TU\textless{ }Topic\ TU\textgreater{ }</code>	<code>/opsx-explore\\TU\textless{ }Topic\ TU\textgreater{ }</code>	<code>/openspec-explore\\TU\textless{ }Topic\ TU\textgreater{ }</code>
propose	Turn the agreed problem into a change proposal and generate its planning artifacts.	<code>/opsx-propose\\TU\textless{ }change-name\ TU\textgreater{ }</code>	<code>/opsx-propose\\TU\textless{ }change-name\ TU\textgreater{ }</code>	<code>/openspec-propose\\TU\textless{ }change-name\ TU\textgreater{ }</code>
apply	Implement the tasks in an existing proposal after reviewing its requirements and design.	<code>/opsx-apply\\TU\textless{ }change-name\ TU\textgreater{ }</code>	<code>/opsx-apply\\TU\textless{ }change-name\ TU\textgreater{ }</code>	<code>/openspec-apply-change\\TU\textless{ }change-name\ TU\textgreater{ }</code>

¹OpenSpec supported tools. The page documents the GitHub Copilot integration, its `.github/skills/` and `.github/prompts/` locations, the `/openspec-apply-change` skill spelling, the `/opsx-apply` command spelling, and the note that Copilot CLI does not read `.github/prompts/`.

²OpenSpec CLI reference. The `init` command accepts a `-V-tools` option for selecting integrations.

³OpenSpec supported tools. The page documents the GitHub Copilot integration, its `.github/skills/` and `.github/prompts/` locations, the `/openspec-apply-change` skill spelling, the `/opsx-apply` command spelling, and the note that Copilot CLI does not read `.github/prompts/`.

OpenSpec action	When to use it	VS Code	IntelliJ IDEA	GitHub Copilot CLI
sync	Merge implemented delta requirements into openspec/specs/ while leaving the change proposal open. Use this when the specification is ready to become the current baseline but the change still needs follow-up work.	/opsx-sync\\TU\ textless{ }change- name\\TU\ textgreater{ }	/opsx-sync\\TU\ textless{ }change- name\\TU\ textgreater{ }	/openspec-sync- specs\\TU\ textless{ }change- name\\TU\ textgreater{ }
archive	Complete a finished change. This merges its deltas into the main specs and moves the change folder into openspec/changes/archive/.	/opsx-archive\\TU\ textless{ }change- name\\TU\ textgreater{ }	/opsx-archive\\TU\ textless{ }change- name\\TU\ textgreater{ }	/openspec-archive- change\\TU\ textless{ }change- name\\TU\ textgreater{ }

C.1. Using the matrix

The VS Code and IntelliJ IDEA entries match because both use the GitHub Copilot IDE integration. OpenSpec generates prompt files under `.github/prompts/`, which the Copilot extension can expose as slash commands in both editors.⁴

GitHub Copilot CLI uses the skill spelling. For example, the CLI equivalent of `/opsx-apply\pets-search` would be `/openspec-apply-change\pets-search`. The IDE command is not expected to resolve in the CLI because the CLI does not read `.github/prompts/`.⁵

You might use the rows as a working loop rather than as mandatory ceremony for every small edit. Explore can help when the problem or existing behavior is unclear. Propose fits once the intended change is understood well enough to record. Apply is available when the proposal is ready to execute. Sync can update the main specification while the change remains open. Archive generally follows verification and completion, when the change is ready to leave the active-work area and its deltas can become part of the lasting specs.⁶

⁴OpenSpec supported tools. The page documents the GitHub Copilot integration, its `.github/skills/` and `.github/prompts/` locations, the `/openspec-apply-change` skill spelling, the `/opsx-apply` command spelling, and the note that Copilot CLI does not read `.github/prompts/`.

⁵OpenSpec supported tools. The page documents the GitHub Copilot integration, its `.github/skills/` and `.github/prompts/` locations, the `/openspec-apply-change` skill spelling, the `/opsx-apply` command spelling, and the note that Copilot CLI does not read `.github/prompts/`.

⁶OpenSpec skills reference and OpenSpec quickstart. These pages define the explore, propose, apply, sync, and archive workflows and their skill names.

The terminal CLI is a separate interface. Commands such as `openspec\init` and `openspec\validate` can manage the project and its artifacts from a shell. Slash commands and skills ask an AI coding tool to carry out a workflow using those artifacts. If a workflow is unavailable, it may help to check that the repository is initialized and valid and that the selected host can see the files it expects.

D. OpenSpec schema.yaml and Template Reference

OpenSpec uses Schemas to configure how an OpenSpec workflow executes. This appendix can help when you need to read or change an OpenSpec workflow. It covers the fields in schema.yaml, the dependency rules between artifacts, and the way templates become generated change files.

D.1. Schema fields

An OpenSpec schema defines a planning workflow in YAML. It names the workflow, lists the artifacts it produces, and describes the handoff from planning to implementation. In the built-in spec-driven schema, the default sequence is explicit: proposal leads to specs and design, and tasks depends on both before implementation can begin.

The top-level fields are:

Field	Type	Required?	Effect
name	string	Yes	Names the schema. OpenSpec uses the schema directory as the lookup key, even if this value differs.
version	positive integer	Yes	Records the schema revision. It does not change OpenSpec behavior by itself.
description	string	No	Describes the workflow when schemas are listed.
artifacts	non-empty list	Yes	Declares the planning artifacts and their dependencies.
apply	mapping	No	Defines what must be complete before implementation starts and, optionally, which task file tracks progress.

Each entry in artifacts defines one document or one set of documents:

Field	Type	Required?	Effect
id	unique string	Yes	Identifies the artifact. Other artifacts and apply use this value in requires.
generates	relative path or glob	Yes	Tells OpenSpec where the artifact is written inside the change folder. For example, specs/**/*.*md can produce one spec per capability.
description	string	Yes	Labels the artifact in the guidance given to the agent.
template	relative path	Yes	Selects the Markdown template under the schema's templates/ directory.
instruction	string	No	Gives the agent artifact-specific guidance, such as whether the document explains why, what, or how.
requires	list of artifact IDs	No	Makes this artifact wait until the named artifacts are complete. The default is an empty list.

It is useful to treat requires as a dependency graph rather than a suggested reading order. OpenSpec marks an artifact ready only after every dependency is complete. It rejects missing IDs, duplicate IDs, and dependency cycles. When several artifacts are ready at once, their order in artifacts determines which one OpenSpec returns first.

The generates value is relative to the active change directory. proposal.md therefore produces openspec/changes/TU\textless{ }change-name\TU\textgreater{ }/proposal.md, while specs/**/*.*md matches Markdown files below that change's specs/ directory. Completion is structural: OpenSpec checks that the expected file exists, or that a glob matches at least one file. The check does not evaluate the document's content.

The optional apply block controls the planning-to-implementation handoff. Its requires field names the artifacts that must exist before apply is ready. Its tracks field points to a relative Markdown file in the change directory, normally tasks.md. OpenSpec uses task checkboxes in that file to report whether work is blocked, ready, or complete. An optional instruction replaces the built-in guidance sent to the implementation agent.

The default workflow has this essential shape:

```

name: spec-driven
version: 1
description: Default OpenSpec workflow - proposal -> specs -> design -> tasks
artifacts:
  - id: proposal
    generates: proposal.md
    description: Initial proposal document outlining the change
    template: proposal.md
    requires: []
  - id: specs
    generates: specs/**/*.*md
    description: Detailed specifications for the change
    template: spec.md
    requires:
      - proposal
  - id: design
    generates: design.md
    description: Technical design document with implementation details
    template: design.md
    requires:
      - proposal
  - id: tasks
    generates: tasks.md
    description: Implementation checklist with trackable tasks
    template: tasks.md
    requires:
      - specs
      - design
apply:
  requires:
    - tasks
  tracks: tasks.md

```

The example omits the longer agent instructions. The fields serve different purposes: description labels the artifact, template supplies its output shape, and instruction explains what the agent should produce. A changed dependency changes the workflow; a changed template changes the starting structure of every new artifact created from it.

D.2. Template structure

A schema directory contains schema.yaml and a templates/ directory. OpenSpec resolves the template value in each artifact entry relative to that directory:

```

openspec/schemas/spec-driven/
├─ schema.yaml
├─ templates/
│   ├─ proposal.md
│   └─ spec.md

```



```

├─ design.md
└─ tasks.md

```

Templates are Markdown skeletons rather than files copied into the change directory. OpenSpec gives the selected template to the agent as its expected output format. The agent fills that structure and writes the result at the path named by generates.

In the default schema, the mapping is:

Artifact	Template	Generated path	Purpose
proposal	proposal.md	proposal.md	Explains why the change is needed, what it changes, its capabilities, and its impact.
specs	spec.md	specs\TU\ textless{ }capability- path\TU\textgreater{ }/ spec.md	Defines observable behavior through requirements and scenarios.
design	design.md	design.md	Explains how the change will be implemented, including decisions, risks, and trade-offs.
tasks	tasks.md	tasks.md	Breaks the implementation into ordered, checkable work items.

The specs entry is the exception to the one-file pattern. It allows a change to produce one delta specification for each capability named by the proposal. The proposal establishes the reason and scope, the spec files define behavior, the design records the technical approach, and the task list turns the plan into work the apply phase can track. Keeping these roles separate helps prevent implementation details from replacing requirements.

When a team needs different documents or a different sequence, it can create a project schema under openspec/schemas\TU\textless{ }name\TU\textgreater{ }/, edit schema.yaml and the matching templates, and run openspec\schema\validate\TU\textless{ }name\TU\textgreater{ } before using it. Forking the built-in spec-driven schema copies its schema definition, templates, and instructions into a project-owned directory. A custom template changes the structure of future artifacts; a schema dependency changes which artifacts can be drafted and when. Built-in schemas ship with OpenSpec, so forking one before editing it is preferable to modifying the package copy.

E. Troubleshooting OpenSpec

E.1. A diagnostic method

For each problem, start with the symptom and a likely cause. Then run the cheapest check that could distinguish between possible causes before choosing a repair. Keep the command output from that check; it may be more useful than repeating a setup step that could change the evidence.

E.2. Installation failures

You might see one of these symptoms:

- openspec cannot be found after npm installation.
- The terminal resolves a different global npm location than expected.
- Windows execution policy or macOS permissions prevent the command from running.
- The installed version differs from the book's 1.13.1 baseline.

E.3. Initialization and update failures

Check whether:

- OpenSpec initialized a different directory than expected.
- Existing generated files conflict with the update.
- The selected GitHub Copilot integration or profile is unavailable.
- Project-owned configuration changed unexpectedly.

E.4. Command discovery failures

You may be dealing with command discovery when:

- VS Code does not show the expected prompt command.
- IntelliJ IDEA does not discover the same workflow.
- A prompt-file command is being confused with an Agent Skill or terminal CLI command.
- The IDE appears to be using an earlier version of the generated integration files.

E.5. Artifact and validation failures

The failure may be in artifact or validation handling when:

- A change cannot identify its schema.
- Artifact dependencies remain blocked.
- Requirements or scenarios fail strict validation.
- A modified requirement does not match the accepted source requirement.
- Archive or sync encounters overlapping deltas.

E.6. Two-repository failures

Compare the repositories when:

- Frontend and backend changes use different business terms.
- The generated Angular client appears stale.
- The OpenAPI document does not match the backend behavior.
- A test passes against a different backend or port than the one you intended.

E.7. Recovery boundaries

Before regeneration, preserve project-owned specs and active changes. Use version control to inspect generated-file changes. If an OpenSpec defect remains reproducible, report the version, operating system, minimal repository state, command, and complete output.

E.8. Online references

- [OpenSpec troubleshooting](#)
- [OpenSpec CLI reference](#)
- [GitHub Copilot troubleshooting](#)

F. Further Reading and Community Resources

OpenSpec is open-source software available through its GitHub project and repository. Those resources include an issue queue and discussions, which can provide information about OpenSpec and Spec-Driven Development more generally.

F.1. Official documentation

The OpenSpec repository and its documentation are useful starting points. The OPSX workflow describes the workflow used throughout this book. The installation guide covers setup, while supported tools and aliases lists the available integrations.

These sources cover the mechanics of using OpenSpec. Your repository's proposal, requirements, design, tasks, and verification evidence still provide the context for a specific change. The documentation can explain how a command works, but it cannot decide whether the change is understood well enough to implement or whether the resulting behavior is correct. Your project team remains responsible for the specs, and OpenSpec is more useful when the team understands them fully.

F.2. Community schemas and extensions

Community-built schemas and extensions may help when the official OpenSpec workflow does not cover a particular business domain or team practice. They can also provide a working example. It is useful to treat each community-built schema as a proposal to evaluate: read its source and examples, check its maintenance activity and compatibility with your OpenSpec version, and identify the assumptions it makes about your repository and development process.

A small, representative change can provide a useful trial before the schema becomes a team convention. It helps to ask whether its artifacts make requirements easier to review, give Copilot useful context, and produce evidence that the implementation matches the intended behavior. If it adds more SDD ceremony without improving those checks, you might cherry-pick the useful ideas from the community-built schema and leave the rest behind. Recording adopted conventions in the repository gives the next developer a way to understand them.

G. Lab Setup

G.1. Required tools

Plan to have these available for each participant:

- Windows 11 or macOS;
- Git and access to the two prepared Bitbucket repositories;
- VS Code or IntelliJ IDEA with GitHub Copilot Chat access;
- OpenSpec 1.13.1;
- Node.js 24.19.0 and npm;
- Java 21 and Maven for the Spring Boot 4.1.1 project;
- Angular 21, Vitest, Playwright, and OpenAPI Generator for the frontend project;
- Quarto only if participants will render the book locally.

Before the class, check the current OpenSpec installation instructions at <https://openspec.dev/docs/installation>. Tool versions and package commands can change.

G.2. Prepare the exercise repositories

Start by cloning the prepared backend and frontend repositories. It may help to confirm that each repository builds and passes its tests before initializing OpenSpec. The backend owns the OpenAPI contract, and OpenAPI Generator produces the TypeScript Angular client used by the frontend.

```
cd commercial-truck-search-api
openspec init

cd ../commercial-truck-search-ui
openspec init
```

When initialization asks which AI tool to configure, select GitHub Copilot. Give each repository its own context instead of copying one configuration into both repositories.

The backend configuration records Java 21, Spring Boot 4.1.1, Maven, Spring Data JPA, H2, test conventions, and ownership of the OpenAPI contract. The frontend configuration records Angular 21, generated-client boundaries, Vitest, Angular component tests, and Playwright.

Review the generated configuration. It can describe stable project facts, but it should not invent database products, deployment platforms, or business rules that the class has not selected.

G.3. Preflight check

Before Morning 1, check that:

1. Both starter applications build and their existing tests pass.
2. openspec\init created the expected project files in both repositories.
3. GitHub Copilot Chat recognizes the expected OpenSpec commands in VS Code and IntelliJ IDEA.
4. Each participant knows which commands belong in the terminal and which belong in chat.
5. Each repository has a clean baseline commit suitable for reset between exercises.

You may want to keep prepared checkpoints for both repositories after each major exercise. That gives participants who miss a step a way to resume from the same accepted specs, contracts, and code state as the rest of the class.

H. Prompt Workbook

Use these prompts as starting points. Replace bracketed decisions with the class's agreed behavior rather than asking the agent to invent them.

H.1. Explore unit-number search

/opsx:explore We are considering an initial commercial truck search capability. Could users search by a 15-character unit number? Please ask about exact versus partial matching, case handling, invalid input, no-match behavior, and the response shown in the UI. Hold off on creating change artifacts until those decisions are clear.

H.2. Propose unit-number search

/opsx:propose\add-unit-number-search Based on the exploration decisions, could you draft the proposal? Please keep implementation choices out of the behavioral requirements and include scenarios for valid input, invalid input, a match, and no match.

H.3. Add VIN search

/opsx:propose\add-vin-search-option Could we add a secondary search mode for a 17-character VIN while preserving unit-number search? Please ask how users would select the mode and how the two modes should share response behavior. Represent the new capability as an additive delta.

H.4. Modify unit-number validation

/opsx:propose\update-unit-number-length We are considering replacing the current unit-number length rule with a maximum of 10 characters. Could you identify every scenario that would change and every existing behavior that should remain intact? Represent the replacement as a modified requirement.

H.5. Update an active filter proposal

/opsx:update Could you revise the active origin-state filter change to include Active Status with values Active and Decommissioned? Please update the proposal, requirements and scenarios, design, and tasks, and flag any unanswered question about how the filters should combine.

H.6. Review generated artifacts

Could you review this OpenSpec change without editing it? Please report contradictions, unsupported assumptions, missing edge cases, tasks that do not trace to requirements, and requirements that lack verification tasks.

H.7. Verify before archive

Could you compare the implemented change with the active delta and task list before archiving? Please identify unmet scenarios, code outside the approved scope, stale tasks, and tests that do not prove the specified behavior. Hold off on archiving the change.

I. Artifact Review Checklists

I.1. Proposal

- Does it make the business problem and intended outcome clear?
- Is the scope narrow enough to review and implement with confidence?
- Does it call out the exclusions that matter for this change?
- Does it leave out technical assumptions that have not been approved?
- Could a reviewer tell whether this adds behavior or modifies what is already accepted?

I.2. Requirements and scenarios

- Does each requirement describe behavior that someone could observe?
- Are terms such as “maximum,” “exact,” and “optional” clear enough to avoid ambiguity?
- Do the scenarios cover successful, invalid, and empty outcomes where those cases matter?
- Would an additive delta leave existing behavior unchanged?
- Does a modified delta include the full replacement requirement?
- Could each scenario become an automated or repeatable test?

I.3. Design

- Does it focus on decisions that have implementation consequences?
- Does it explain the API, UI, validation, and persistence boundaries that may be affected?
- Does it identify compatibility or migration concerns that reviewers may need to consider?
- Does it record tradeoffs instead of presenting one choice as inevitable?
- Does it keep from duplicating behavioral requirements?

I.4. Tasks

- Would each task produce a visible code, test, or documentation result?
- Could each requirement be traced to implementation and verification work?
- Are the tasks ordered in a way that reflects their dependencies?
- Are the review and test steps clear enough to follow?
- Would a new agent session be able to tell what remains incomplete?

I.5. Apply review

- Does the implementation appear to match the latest artifacts rather than an earlier chat message?
- Are the frontend and backend validation rules consistent with one another?
- Do the tests demonstrate the scenarios rather than only executing lines?
- Has the agent changed code outside the approved scope?
- Does the task list reflect what has actually been completed?

I.6. Archive readiness

- Has the team accepted the final behavior?
- Do the affected tests pass?
- Would the main spec remain coherent after the delta merges?
- Have overlapping active changes been reviewed against the new baseline?
- Is any unfinished work represented by a separate tracked change?

J. Instructor Guide

J.1. Course shape

Each morning is planned for 90 minutes and follows the same rhythm: establish one concept, demonstrate one artifact transition, let participants perform it, and review the resulting evidence.

Time	Morning 1	Morning 2	Morning 3
0-15 min	Context continuity and SDD	Brownfield baseline	Parallel-change risks
15-30 min	OpenSpec structure and setup	Additive vs. modified deltas	Worktrees and isolation
30-55 min	Explore and propose	Add VIN search	Propose and update filters
55-75 min	Apply and test	Modify unit-number length	Apply, sync, and resolve overlap
75-90 min	Archive and debrief	Archive and compare specs	Governance review and wrap-up

J.2. Teaching checkpoints

It may help to measure progress by more than whether the agent generated files. Pause at these points:

1. After exploration, ask which ambiguities a person resolved.
2. After proposal, invite participants to separate behavior from implementation.
3. After apply, consider tracing one scenario through UI, API, persistence, and test code.
4. After archive, read the main spec without referring to the completed delta.

J.3. Expected friction

Participants may type slash commands into the terminal, accept generated assumptions, or treat task checkmarks as test results. Keeping the terminal/chat distinction visible can help, as can asking for an artifact review before each apply or archive step.

Tool output may differ across OpenSpec releases or coding-agent models. Teach the invariant workflow and review criteria rather than requiring identical generated prose.

J.4. Recovery points

Maintain three instructor baselines:

- **Baseline A:** Initialized project with no truck-search spec.
- **Baseline B:** Archived unit-number search.
- **Baseline C:** Archived VIN search and 10-character unit-number rule.

These baselines can keep a setup problem from consuming the next morning. They also let the instructor demonstrate a delta against a known source of truth.

J.5. Closing discussion

Ask participants to name one change in their own codebase where conflicting interpretations would cost more than writing a short spec. Then ask for one change too small or reversible to justify the workflow. The contrast can help keep SDD grounded as an engineering tradeoff rather than a universal ceremony.